

基于改进 SA-ALNS 的多车型 VRPTW 优化及限行成本与碳排放评估

摘要

同城配送是城市物流体系中的关键组成部分，其实际运行过程中往往处于动态且高度不确定的环境中。现有调度模型目标优先级逻辑混乱，且应对多类突发扰动的适应能力不足，难以兼顾合规管控、成本控制与低碳减排等多重现实需求。针对上述问题，本文在能耗测算中引入载重率修正系数，以提高估算准确性，同时采用字典序分层方法协调多目标优化关系，在此基础上构建面向多类突发场景的动态重调度模型，从而提升**配送调度**的综合应用效果。

针对问题一，围绕多车型、多约束条件下的车辆路径优化需求，本文以**整体配送总成本最小**为目标，结合载重限制、路线连通、订单履约等多重约束，构建**多车型 VRPTW 优化模型**，并采用改进 **SA-ALNS 自适应邻域搜索算法**进行求解。结果显示：所得调度方案共启用车辆 **73 辆**、规划配送线路 **121 条**，整体配送总成本 **85455.83 元**，其中固定成本 29200 元、能耗成本 26642 元、碳排放 7954 元、时间窗成本 11660 元，订单全覆盖且各项约束条件均严格满足。该模型有效压缩运营开支，统筹配送效率与低碳目标，完成运力资源精细化配置，为常规物流场景车辆调度提供科学可行的优化方案。

针对问题二，结合绿色配送区限行政策，本文在问题一构建的模型基础上进行优化改进：以**字典序多级优化目标**替换原单一目标函数，在原有约束基础上新增空间、时间、车型三约束，确保方案符合限行规则；同时对 SA-ALNS 算法进行优化，增加**合规性校验**并融入错峰配送策略，最终得到满足限行要求的最优调度方案。结果表明，与问题一相比，政策约束下总成本由 **85455.83 元**增至 **94849.17 元**，增幅 **10.99%**；启用车辆数由 **73 辆**增至 **83 辆**，其中燃油车占比上升、新能源车数量略有下降；碳排放总量由 **12237 kg**增至 **12694 kg**，整体增幅可控。改进后的模型在严格遵守限行政策的同时，实现了运营成本、配送效率与碳排放的协同权衡，为城市绿色物流管控场景下的车辆调度提供了科学、可行的决策支撑。

针对问题三，本文首先构建**单一事件扰动**下的即时响应机制，实现突发情况下路径与运力的快速调整；在此基础上，进一步面向多事件并发生的复杂情况，构建**组合事件优先级处理模型**，根据扰动的影响程度与紧急程度确定响应顺序，从而避免多扰动叠加导致的调度混乱与决策失效。以“订单取消+新增订单”的典型**组合扰动场景**为例进行仿真结果说明：相较扰动发生前，系统总成本与碳排放量均有明显下降，且模型响应时间为**9.8ms**。

最后，本文结合绿区半径等关键参数开展**灵敏度分析**。通过量化分析参数波动带来的成本、调度策略及系统稳定性变化，进一步验证模型的合理性与鲁棒性，为现实配送问题的参数选取与方案优化提供理论参考与实践依据。

关键词：配送调度；多车型 VRPTW 优化模型；SA-ALNS 算法；多场景仿真；灵敏度分析

一、问题重述

1.1 问题背景

某省会城市头部物流企业依托混合动力车队承担大量订单配送任务，覆盖多个客户点与订单，其中部分客户位于绿色配送区域内。各客户均对货物重量、体积及交付时间有明确要求。企业配备五类燃油与新能源车辆，其行驶速度受交通时段影响。同时，能耗与车速呈 型相关，碳排放可由能耗线性折算。在静态运营、限行政策及动态突发事件等场景下，企业亟需统筹客户满意度、运营成本与环保效益，构建科学合理的绿色配送调度模型，实现低成本、高效率与低排放的可持续配送。

1.2 问题提出

问题一：以配送总成本最低为优化目标，综合考量车辆载重与容积上限、交付时间以及车速时变等多重现实约束，构建车辆调度模型。通过模型求解输出最优调度方案。

问题二：在问题一构建的车辆调度模型的基础上，加入严禁燃油车驶入绿色配送区这一限行政策后再重新配送调度路径，并说明该限行政策对调度成本、车辆分配及碳排放的影响。

问题三：构建动态调度优化策略，使其能够对订单取消、配送地址变更等突发事件及时做出响应，实现配送路径与车辆方案的实时调整。

二、问题分析

2.1 问题一分析

问题一聚焦多车型静态配送调度优化，可拆解为以下子问题：

子问题一：在不同车型载重、容积限制下，为订单匹配合适车型，实现需求全覆盖。

子问题二：在软时间窗约束下，设计车辆行驶路径与服务顺序，平衡时效与成本。

子问题三：在基础约束下，协调不同车型的能耗、固定成本与碳排放，寻求兼顾配送成本与服务效率的整体最优方案。

2.2 问题二分析

问题二聚焦限行政策下的合规配送调度优化，本文将其拆分为三个子问题进行分析：

子问题一：明确限行时段内绿色配送区的通行规则，缕清燃油车与新能源车的服务边界与限制条件。

子问题二：识别各路线的通行风险，明确非限行时段的服务窗口要求。

子问题三：在满足限行约束的前提下，调整车型使用结构与路径方案，实现配送成本与碳排放的协同优化。

2.3 问题三分析

问题三旨在解决多重突发扰动下的动态配送调度难题，本文将其拆解为两个子问题：

子问题一：分析订单取消、配送地址变更、时间窗口调整以及新增订单四类独立扰动的调度逻辑与相互影响，明确并发场景下的潜在冲突风险。

子问题二：基于单事件扰动调度模型，制定多事件并发的协同解决策略，在满足各项基础约束的前提下，实现全局运营总成本最优。

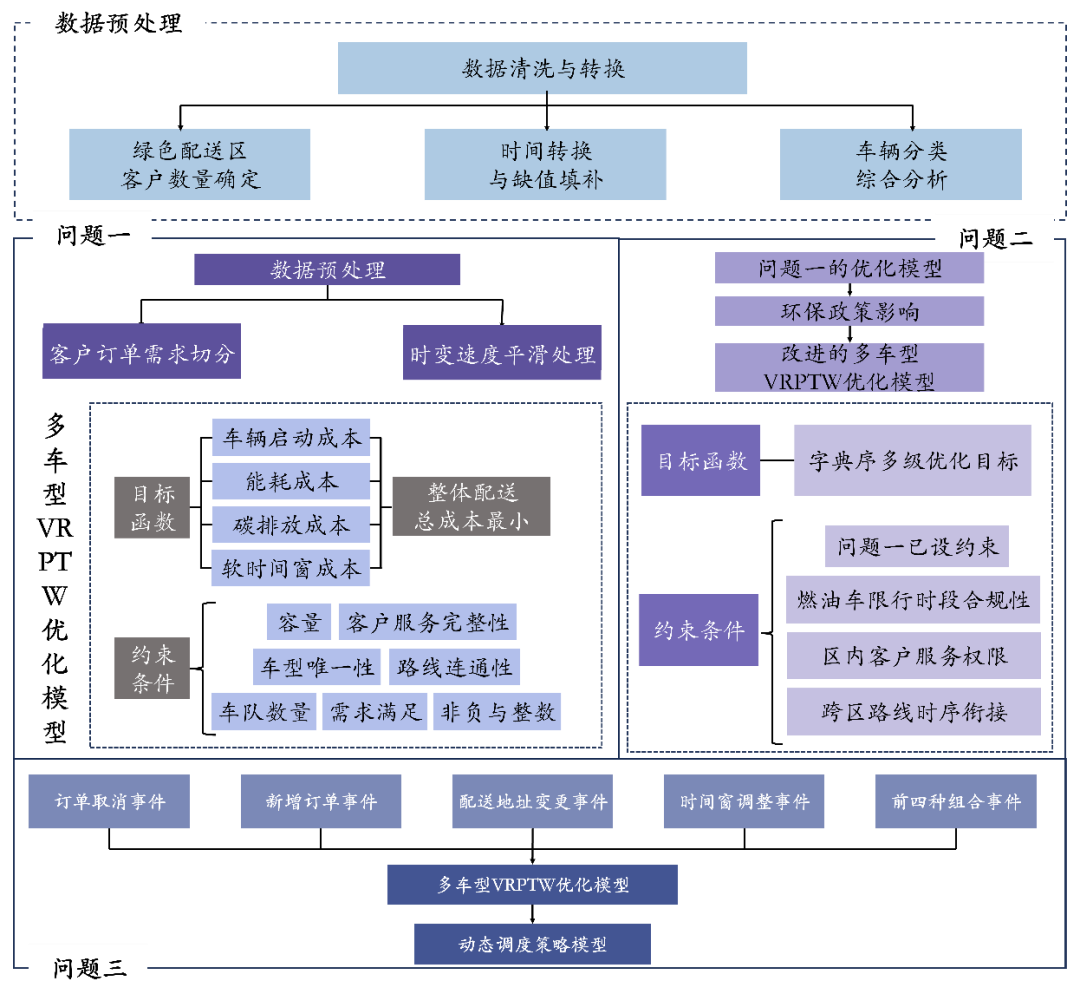


图 1 整体思路框架

三、模型假设

- 1. 假设所有车辆从同一配送中心出发并最终返回，配送中心坐标为原点，编号为节点 0。
- 2. 假设每个客户的卸货服务时间固定为 20 分钟，与需求量无关，不考虑装卸过程中的随机延误。
- 3. 假设各时段行驶速度正态分布的期望值能够作为规划建模的代表速度，速度的随机性保留在后续灵敏度分析中处理。
- 4. 假设时段切换边界用 15 分钟线性过渡能消除速度不连续跳变对旅行时间积分的影响。

5. 假设车辆行驶能耗按区段初始载重线性修正，满载比空载能耗高出固定比例。
6. 假设同一客户的不同批次配送视为独立交货事件，时间窗成本按各批次中最早到达时刻计算早到成本，以最晚到达时刻计算晚到成本，两者共同决定违约费用。
7. 假设同一车辆完成一趟配送后，在配送中心经过 20 分钟周转，即可立即承接下一趟次，无其他时间约束。
8. 假设扰动发生后，车辆在途调整路线的成本仅按新增里程与时间计算，无额外固定惩罚成本。

四、符号说明

这里只列出部分全局参数，其他参数见文中：

符号	含义	单位
$N = \{1, 2, \dots, 98\}$	客户集合	-
$M = \{0\} \cup N$	所有节点构成集合	-
$K = \{A1, A2, A3, B1, B2\}$	车辆集合	-
q_i	客户 i 的重量需求	kg
v_i	客户 i 的体积需求	m^3
$[a_i, b_i]$	客户 i 的软时间窗	-
d_{ij}	客户 i 到客户 j 的道路距离	km
t_{ik}	车辆 k 到达客户 i 的时间	min
η	油耗碳排放系数	-
γ	电耗碳排放系数	-

五、数据预处理

5.1 数据清洗

在对所给数据进行清洗时发现表格“订单信息”中存在缺失值，为保证后续分析顺利进行，根据以下步骤对缺失值进行插值处理。

Step1: 计算含缺失值的客户的所有货物容重比 $\omega = q / u$ ；

Step2: 以同一客户的 ω 中位数作为其货物容重比基准值，记为 ω_0 ；

Step3: 计算重量缺失值 q' ： $q' = u \times \omega_0$ ； 计算体积缺失值 u' ： $u' = q / \omega_0$ 。

以客户 4 为例，其“订单信息”插值处理结果如下：

表 1 客户 4 插值处理结果

订单编号	重量	体积	客户 ID	缺失重量数据	缺失体积数据
11	169.07	0.33	4	FALSE	FALSE
17	10.50	0.02	4	FALSE	FALSE
61	36.80	0.08	4	FALSE	TRUE

70	316.08	0.68	4	FALSE	FALSE
136	26.50	0.06	4	FALSE	FALSE
148	18.22	0.15	4	FALSE	FALSE
187	93.92	0.32	4	FALSE	FALSE
216	18.40	0.04	4	FALSE	FALSE
221	29.25	0.06	4	FALSE	FALSE
243	6.40	0.02	4	FALSE	FALSE
271	23.40	0.17	4	FALSE	FALSE
282	695.02	1.56	4	FALSE	FALSE

5.2 数据转换

1.绿色配送区客户数量确定

绿色配送区是指距市中心 10 公里的圆形区域内。计算各客户点与市中心的距离，若距离小于 10 公里，则该客户点在绿色配送区内。最终，统计得前十五位客户恰好在绿色配送区内，结果如表 2 所示：

表 2 绿色配送区客户确定

客户 ID	X(km)	Y(km)	距市中心距离	是否在绿色配送区
1	0.75	6.13	6.17	TRUE
2	3.49	-2.10	4.07	TRUE
3	-2.79	-7.07	7.60	TRUE
4	-5.53	4.97	7.44	TRUE
5	-7.59	-3.95	8.55	TRUE
6	-4.66	-7.51	8.84	TRUE
7	-8.51	3.40	9.16	TRUE
8	2.32	3.18	3.94	TRUE
9	4.21	7.39	8.50	TRUE
10	-5.87	-5.57	8.09	TRUE
11	0.52	-2.38	2.44	TRUE
12	1.22	-1.35	1.82	TRUE
13	1.95	8.96	9.17	TRUE
14	-0.94	-8.47	8.52	TRUE
15	-1.70	-3.10	3.54	TRUE

同时通过对客户地理分布及其距离分布的可视化，能够直观呈现配送网络的空间结构特征，清晰展示客户在绿色配送区内、外的分布格局、距离梯度及订单量，为后续限行政策下的车辆调度与路径规划提供数据基础与空间依据。具体如图 2 所示。

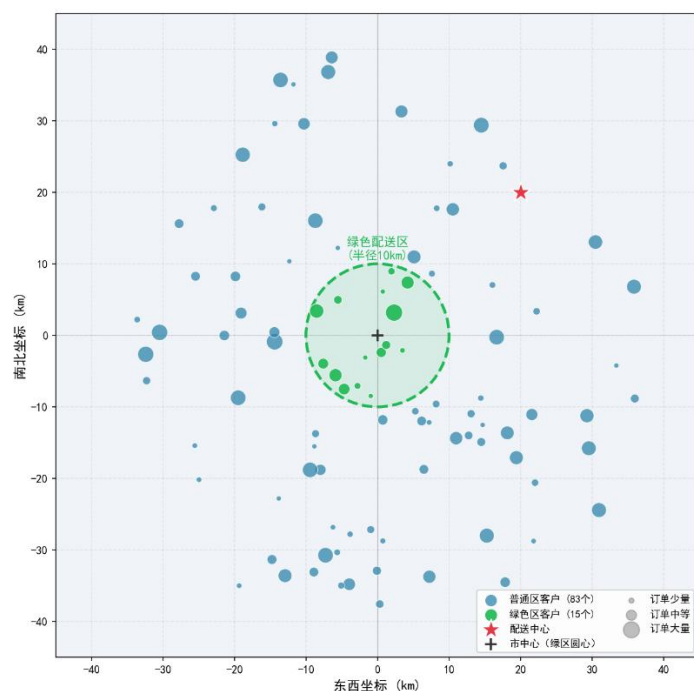


图 2 客户地理分布图

2.时间转换

以 00:00 为起始时间，将各客户的软时间窗口节点转化为分钟，便于后续计算。由于篇幅有限，这里仅展示前五位客户的时间转换结果，如表 3 所示。

表 3 前五位客户软时间窗口时间节点转换

客户 ID	开始时间	结束时间	距开始时长	距结束时长
1	11:33	12:22	693	742
2	17:53	18:49	1073	1129
3	14:33	15:21	873	921
4	19:29	20:37	1169	1237
5	19:28	20:26	1168	1226

3.车辆类型综合分析

为明确不同车型的功能定位与适配场景，本文对燃油车与新能源车的载重、容积、车辆数量及启动成本等核心参数进行了标准化处理与对比分析，结果如图 3 所示。结果表明，燃油车 A 数量充足，是车队的主力车型；新能源车 A 重载大容积优势显著但数量有限，是限行时段绿色配送区服务的核心支撑；各类车型呈现出明显的差异化互补特征，为后续限行政策下的车型分配与路径规划提供了关键依据。

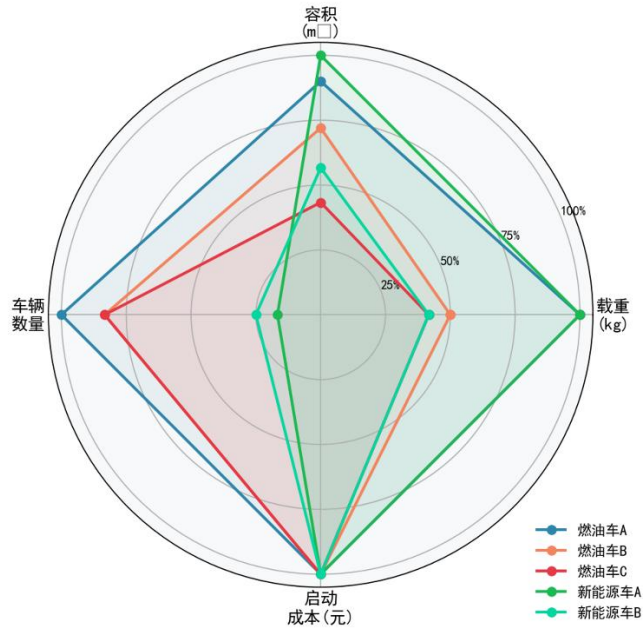


图 3 车辆类型综合参数雷达图

六、模型建立与求解

6.1 问题一模型建立与求解

6.1.1 问题一数据预处理

为保证问题一模型求解的可行性、效率与结果真实性，本文首先对清洗好的数据进行问题一数据预处理，主要包括客户订单需求切分与时变交通速度平滑处理两部分。

一方面，针对部分客户订单需求超过车辆最大容量的问题，通过需求切分算法将超大规模订单拆分为多个满足单车载重与容积约束的子需求块；另一方面，针对题目分段常速模型在时段交界点存在速度突变的问题，构建带线性过渡窗口的平滑时变速度函数，为跨时段路段行驶时间的准确计算提供支撑。

6.1.1.1 客户订单需求切分

由于不同客户 i 对重量及体积的需求不同，本文采用需求切分算法将部分客户的需求切分为多份，以适配车辆的载重和容积限制，该算法主要包括需求切分与子需求块生成两个部分，具体如下：

Step1: 客户需求切分

原始客户订单数据中，部分客户的重量或体积需求可能超过单辆车的最大承载能力，导致订单无法被一次性服务。为此，本文引入均分切分机制^[1]，将超大规模订单拆分为多个满足车辆载重与容积约束的子需求块，以适配多车型车辆的装载能力。

设客户 i 的重量与体积需求分别为 q'_i 、 u'_i ，则其需求切分的份数 e_i 为：

$$e_i = \max \left(1, \left\lceil \frac{q_i'}{Q_{ik}} \right\rceil, \left\lceil \frac{u_i'}{V_{ik}} \right\rceil \right) \quad (1)$$

其中 Q_{ik} 、 V_{ik} 表示装载客户 i 的 k 类车的最大载货量、最大体积量。

当客户需求重量和体积不超过装载它的车辆的最大承载时， $e_i = 1$ ，即无需切分；当任一维度（重量或体积）超过承载时，取两个取整结果中的较大值，保证切分后的每一份子需求均满足车辆容量约束。

Step2: 子需求块生成与排序

根据切分的份数 e_i 将客户 i 的总需求均匀分配到每个子需求块中，即为：

$$q_{ip}' = \frac{q_i'}{e_i}, \quad u_{ip}' = \frac{u_i'}{e_i} \quad p = 1, 2, \dots, e_i \quad (2)$$

其中 q_{ip}' 和 u_{ip}' 分别为客户 i 第 p 个子需求块的重量与体积。

由于后续将采用“大件优先”的装载策略，因此本文进一步将所有的子需求块按照重量降序、再按体积降序排列。利用该排列方式能够优先处理高重量和高体积的需求块，有利于提高车辆的载重与容积利用率。

6.1.1.2 时变交通速度平滑处理

由于题目中仅给出部分时段及其对应速度分布，其它时段并未给出速度分布，且在时段外仍有订单需要配送，因此本文参考曹庆奎结论，考虑行车速度实际情况以及保证与已给时段速度分布保持一致，对整天各个时段速度分布都进行了分段。

同时题目给出的各时段速度均服从正态分布^[2]，故为简化同一时段行驶时间的计算，本文取各时段速度分布的均值作为该时段的标准行驶速度，既保证了行驶时间计算的合理性，也有效降低了建模复杂度。其不同时段速度函数如下：

$$v(m) = \begin{cases} 55 \text{ km/h} & m \in [0, 360) \cup [540, 600) \cup [780, 900) \cup [1200, 1440) \\ 35.4 \text{ km/h} & m \in [360, 480) \cup [600, 690) \cup [900, 1020) \cup [1080, 1200) \\ 9.8 \text{ km/h} & m \in [480, 540) \cup [690, 780) \cup [1020, 1080) \end{cases} \quad (3)$$

在构建好不同时段速度函数后，本文考虑，若以开车时间所落在的时段作为该段的行驶速度并不够合理，在时段交界点存在速度突变，产生跨时段误差。如车辆在前一时段末发车，但大部分行驶时间实际落在下一时段，此时则会产生显著偏差。为此，本文进一步对分段速度函数进行改进，在每个时段的切换点 b 前后设置长度为 $\Delta = 15 \text{ min}$ 的线性过渡区间，使速度从前一时段均值平滑过渡到后一时段均值，消除速度突变。

设时段切换点集合为 $B = \{360, 480, \dots, 1200\}$ ，对于任意 $b \in B$ ，当 $m \in [b - \Delta, b + \Delta)$ 时，带线性过渡窗口的全天分段折线速度函数为：

$$v(m) = \begin{cases} v_{\text{前}}(b) + (v_{\text{后}}(b) - v_{\text{前}}(b)) \cdot \frac{m - (b - \Delta)}{2\Delta} & m \in [b - \Delta, b + \Delta) \\ v_{\text{段}}(m) & \text{其它时段} \end{cases} \quad (4)$$

其中 $v_{\text{前}}(b)$ 、 $v_{\text{后}}(b)$ 分别为切换点前后时段的标准速度， $v_{\text{段}}(m)$ 为原分段常速的时段速度。

其具体全天分段的速度变化趋势如下图 4 所示：

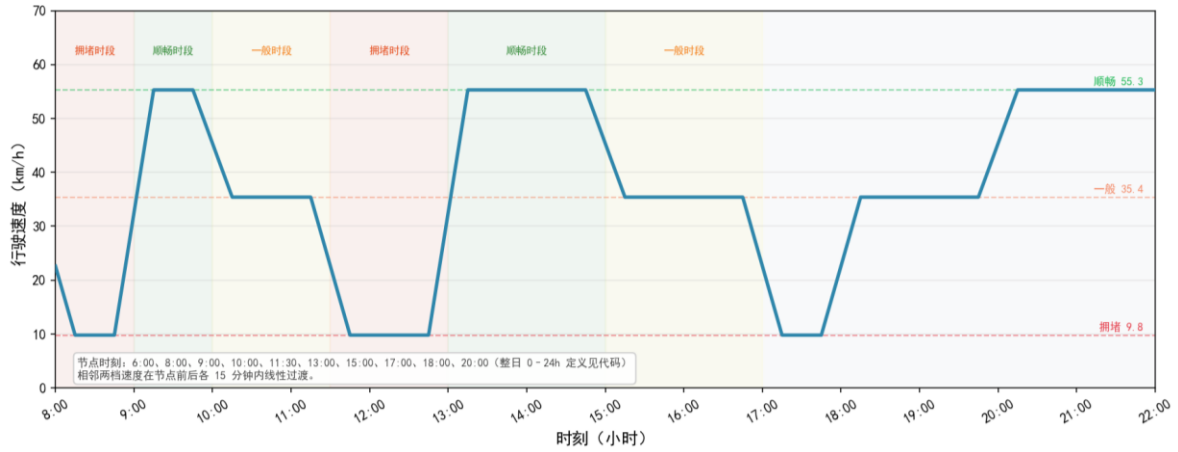


图 4 全天分段折线速度函数示意图

若出现跨天时刻，即 $(m \geq 1440 \text{ 或 } m < 0)$ ，统一按 $m \bmod 1440$ 映射到当日分钟轴再计算。改进之后既保留了原模型的各时段特征，又有效降低了跨时段路段的行驶时间计算误差。

6.1.2 问题一模型建立

为合理规划多车型配送路线与客户服务次序，本文针对静态环境下的绿色物流调度需求，构建多车型 VRPTW 优化模型。该模型以整体配送总成本最小为目标，在满足客户订单需求的基础上，综合考虑车辆固定启动、载重影响下的能耗波动、客户时间窗约束以及碳排放成本，通过完整约束体系求解最优调度方案。

同时为提升模型的合理性与现实贴合度，本文将其模型进一步改进优化：

一是由于题目中仅给出满载时能耗比空载时高，即：燃油车满载高 40%，新能源车满载高 35%。但仅一个确定的数值并不能包含车辆在行驶过程中的所有实际情况，即车辆行驶时的实际能耗应当受载货量的非线性影响，因此本文认为满载为车辆装载满（重量达到限制），其余情况引入载重率修正系数，动态调整能耗增加比例，其公式如下：

$$\delta_{ik} = 1 + \theta_k \cdot \min\left(1, \frac{\rho_{ik}}{Q_k}\right) \quad (5)$$

其中 θ_k 为满载修正系数，即燃油车为 0.4，电车为 0.35； ρ_{ik} 为车辆 k 离开客户 i 时的载重量， Q_k 为第 k 类车的最大载货量。

二是结合现实物流运营模式，车辆仅需支付一次启动固定费用，且不再限制单车仅执行单次配送任务。本文允许同一车辆完成多趟循环配送任务，依据实际投入的车辆数量核算固定成本，有效规避传统模型中单车单线分配的理想化缺陷，提升调度方案的实际可行性。

基于上述优化，本文构建具体模型如下：

1.决策变量：

决策变量是模型优化求解的核心，用于刻画配送过程中的调度决策行为，明确各变量的取值范围与物理意义，确保决策结果的可行性与合理性，具体定义如下：

$x_{ijk} \in \{0,1\}$ ：二进制决策变量，用于表示车辆 k 是否从客户 i 行驶到客户 j 。当 $x_{ijk} = 1$ 时，说明车辆 k 选择行驶路段 (i,j) ；当 $x_{ijk} = 0$ 时，说明车辆 k 不选择行驶路段 (i,j) ，该变量直接决定车辆的配送路径。

$m_k \in \{0,1\}$ ：二进制决策变量，用于表示车辆 k 是否启用：当 $m_k = 1$ 时，说明配送中心启用车辆 k ；当 $m_k = 0$ 时，说明车辆 k 未被启用，该变量直接关联车辆固定成本的计算。

$z_{ri} \in \{0,1\}$ ：二进制决策变量，用于表示路线 r 是否服务客户 i ；当 $z_{ri} = 1$ 时，说明路线 r 服务客户 i ；当 $z_{ri} = 0$ 时，说明路线 r 未服务客户 i 。

$t_{ik} \geq 0$ ：连续决策变量，用于表示车辆 k 到达客户 i 的时刻。

2.目标函数：

该模型以整体配送总成本最小为目标，其总成本由车辆固定启动成本、能耗成本、碳排放成本以及软约束时间窗成本四类构成，其目标函数表达式如下：

$$\min C = \sum_{i=1}^4 C_i \quad (6)$$

(1) 车辆固定启动成本 C_1 ：每启用一辆配送车辆，仅计算一次固定成本；车辆启动后，即使多次往返、重复载货开展多趟配送，也不再重复累加固定费用。其公式如下：

$$C_1 = \sum_{k \in K} m_k \cdot F_k \quad (7)$$

其中 F_k 表示车辆 k 的启动成本，因所有车辆启动成本相等，即 $F_k = 400$ 。

(2) **能耗成本** C_2 ：车辆在配送过程中因行驶产生的燃油或电能消耗，进而转换为的金额。由于需考虑题目中提出的早到等待、晚到惩罚^[3]以及在规定时间内送达三种情况，本文针对三种情况分析，定义车辆 k 服务完客户 i 后前往下一个客户的开始时间为：

$$t'_{ik} = \begin{cases} a_i + t_0 & \text{早到等待} \\ t_{ik} + t_0 & \text{晚到惩罚} \end{cases} \quad (8)$$

其中 $[a_i, b_i]$ 为客户 i 的软时间窗。

那么车辆 k 从客户 i 到客户 j 的能耗为：

$$C_{ijk}^{eng} = \begin{cases} FPK(v(t'_{ik})) \cdot \frac{d_{ij}}{100} \cdot c_1 \cdot \delta_{ik} & k \in F \\ EPK(v(t'_{ik})) \cdot \frac{d_{ij}}{100} \cdot c_2 \cdot \delta_{ik} & k \in E \end{cases} \quad (9)$$

其中 d_{ij} 为从客户 i 到客户 j 的道路距离； c_1 为燃油车每升油费； c_2 为新能源车每度电费； $FPK(v)$ 、 $EPK(v)$ 为油耗、电耗与车速的 U 型关系。

因此，能耗成本为：

$$C_2 = \sum_{k \in K} \sum_{i \in M} \sum_{j \in M, j \neq i} x_{ijk} \cdot C_{ijk}^{eng} \quad (10)$$

(3) **碳排放成本** C_3 ：车辆配送过程中油耗或电耗转换为碳排放，进而碳排放转换为的金额。其公式如下：

$$C_3 = c_3 \cdot \left(\eta \cdot \sum_{k \in K} \sum_{i \in M} \sum_{j \in M, j \neq i} H_{ijk}^{eng} + \gamma \cdot \sum_{k \in K} \sum_{i \in M} \sum_{j \in M, j \neq i} G_{ijk}^{eng} \right) \quad (11)$$

其中油耗为 $H_{ijk}^{eng} = FPK(v(t'_{ik})) \cdot \frac{d_{ij}}{100} \cdot \delta_{ik}$ ；电耗为 $G_{ijk}^{eng} = EPK(v(t'_{ik})) \cdot \frac{d_{ij}}{100} \cdot \delta_{ik}$ ； η 、 γ 为油耗、电耗的碳排放转换系数。

(4) **软时间窗成本** C_4 ：当车辆服务时间超出客户约定的时间窗时，需承担的惩罚成本。本文针对不同情况，得到其早到或晚到的时间表达式。

在早到等待情况下，车辆 k 早到客户 i 的时间为：

$$E_t^+ = \max \left(\left\lfloor \frac{a_i - t_{ik}}{60} \right\rfloor + 1, 0 \right) \quad (12)$$

其中因后续计算软时间窗约束成本时，单价是以小时计费，故仅 E_t^+ 单位为小时。

在晚到惩罚情况下，车辆 k 晚到客户 i 的时间为：

$$E_i^- = \max\left(\left\lceil \frac{t_{ik} - b_i}{60} \right\rceil + 1, 0\right) \quad (13)$$

其中 E_i^- 单位为小时。

因此，软时间窗成本为：

$$C_4 = \sum_{i \in N} (C^+ \cdot E_i^+ + C^- \cdot E_i^-) \quad (14)$$

其中 C^+ 为早到单位时间等待成本； C^- 为晚到单位时间等待成本。

3.约束条件：

(1) 需求满足约束：客户 i 的重量需求与体积需求，需要被所有服务该客户的路线完整满足，允许由多条路线拆分完成配送。其公式为：

$$\begin{cases} \sum_{r \in R_k} q_{ri} = q_i' \\ \sum_{r \in R_k} u_{ri} = u_i' \end{cases}, \quad \forall i \in N \quad (15)$$

其中 q_{ri} 、 u_{ri} 分别为路线 r 为客户 i 提供的重量配送量、体积配送量。

(2) 客户服务完整性约束：确保每一位客户 i 都至少被一条路线服务，与需求满足约束形成闭环，避免出现没有任何路线服务客户 i 。其公式为：

$$\sum_{r \in R_k} z_{ri} = 1, \quad \forall i \in N \quad (16)$$

(3) 容量约束：每条路线 r 上，所有客户的重量需求之和不能超过所用车型的载重限制，体积需求之和不能超过所用车型的容积限制。其公式为：

$$\begin{cases} \sum_{i \in N} q_{ri} \leq Q_{kr} \\ \sum_{i \in N} u_{ri} \leq V_{kr} \end{cases}, \quad \forall r \in R_k \quad (17)$$

其中 Q_{kr} 、 V_{kr} 分别为路线 r 所用车型 K 的最大载重、最大容积。

(4) 路线连通性约束：车辆在客户节点的驶入次数与驶出次数相等，保证路径连续；同时每条路线都应该从配送中心出发并最终返回配送中心，形成闭环。其公式为：

$$\begin{cases} \sum_{j \in V'} x_{ijk}^r = \sum_{j \in V'} x_{jik}^r, & \forall i \in N, r \in R_k \\ \sum_{j \in N} x_{0jk}^r = \sum_{j \in N} x_{j0k}^r = 1, & \forall r \in R_k \end{cases} \quad (18)$$

(5) **车型唯一性约束**: 每条路线 r 仅能由一种车型执行, 即一条路线中不存在多车型混用的情况。其公式为:

$$\sum_{k \in K} m_k^r = 1, \quad \forall r \in R_k \quad (19)$$

(6) **车队数量约束**: 车型 k 的实际启动车辆数不能超过配送中心可调配该车型车辆总数的限制。其公式为:

$$P_k' \leq P_k, \quad \forall k \in K \quad (20)$$

其中 P_k 为 k 类车辆的最大可用数量; P_k' 为 k 类车辆实际占用的车辆数量。

(7) **非负与整数约束**: 重量、体积、时间等连续变量均取非负值; 路径决策变量 x_{ijk}^r 与车型选择变量 m_k^r 为二进制变量, 仅取 0 或 1。其公式为:

$$\begin{cases} q_{ri}, u_{ri}, t_{ik}, t_{ik}' \geq 0 \\ x_{ijk}^r, m_k^r \in \{0, 1\} \end{cases} \quad (21)$$

综上可得, 该多车型 VRPTW 优化模型为:

$$\begin{aligned} \min C &= \sum_{i=1}^4 C_i \\ \left\{ \begin{array}{ll} \sum_{r \in R_k} q_{ri} = q_i', & \forall i \in N \\ \sum_{r \in R_k} u_{ri} = u_i', & \forall i \in N \\ \sum_{r \in R_k} z_{ri} = 1, & \forall i \in N \\ \sum_{i \in N} q_{ri} \leq Q_{kr}, & \forall r \in R_k \\ \sum_{i \in N} u_{ri} \leq V_{kr}, & \forall r \in R_k \\ \sum_{j \in V'} x_{ijk}^r = \sum_{j \in V'} x_{jik}^r, & \forall i \in N, r \in R_k \\ \sum_{j \in N} x_{0jk}^r = \sum_{j \in N} x_{j0k}^r = 1, & \forall r \in R_k \\ \sum_{k \in K} m_k^r = 1, & \forall r \in R_k \\ P_k' \leq P_k, & \forall k \in K \\ q_{ri}, u_{ri}, t_{ik}, t_{ik}' \geq 0 \\ x_{ijk}^r, m_k^r \in \{0, 1\} \end{array} \right. \quad (22) \end{aligned}$$

6.1.3 问题一模型求解与分析

问题一构建的模型本质上是一个带复杂约束的大规模优化问题，其求解难度主要源于时变速度、非线性能耗与可拆分订单、车辆复用带来的组合复杂性。由于问题规模较大，直接求解全局最优不具备工程可行性，本文采用“初始可行解构造+局部搜索优化”的两阶段思路，兼顾解质量与求解效率。

1.初始可行解构造

Step1: 在各车型保有量上限约束下，通过枚举快速筛选出满足总重量、总体积需求的最少车辆组合，并优先选择燃油车数量更少的方案，为后续合规性调度预留空间。

Step2: 采用贪心策略，按客户订单的“密度比”降序处理，将客户需求分批装入已选定的车辆中，在满足载重与容积约束的前提下，完成订单拆分与任务分配。

Step3: 对每辆车分配的客户节点，采用贪心最近邻算法确定访问顺序，从配送中心出发，每次选择距离当前位置最近的未访问节点，快速形成初始闭合路径。

Step4: 对已生成的路径，以首个客户的时间窗起点为目标，遍历全天候选发车时刻，选择使早到量最小、时间偏差次小的发车时间，初步满足时间窗约束。

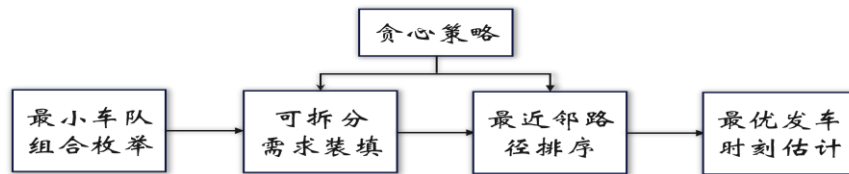


图 5 初始可行解

2.局部搜索优化

本文进一步采用模拟退火驱动的自适应大邻域搜索算法（SA-ALNS），以初始可行解为基础，通过多种调整操作持续优化配送路线与发车时间，同时利用温度衰减策略平衡优化过程。后续再通过路线压缩进一步减少车辆使用数量，在保证方案可行的前提下，显著降低整体配送成本，为问题一提供了一套最优配送方案。其算法流程图如下：

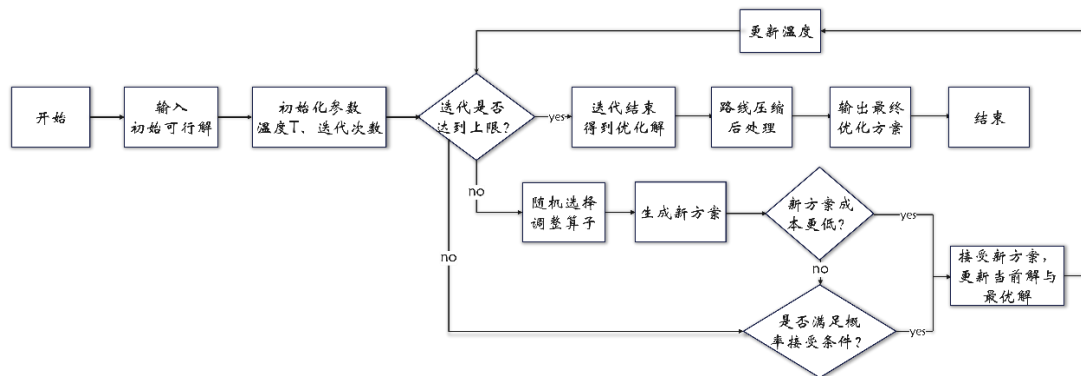


图 6 SA-ALNS 算法流程图

经过两阶段求解框架的迭代优化，算法在保证求解效率的前提下，有效降低了初始可行解的运营成本，同时全程严格满足了所有约束条件。为了更直观地呈现优化效果、验证方案的工程可行性，下面本文先给出问题一的最终调度结果，再从多个维度，对最终调度方案的关键指标与整体构成进行详细分析。

本次调度方案的核心结果如下表 4 所示：

表 4 问题一核心结果

总配送成本/ 元	总行驶里程/公 里	启用车辆总数/ 辆	形成配送路线/ 条	碳排放总量/千 克
85455.83	16748.76	73	121	12237

其中所有客户订单需求均100%覆盖，无遗漏；同时载重、容积、路径、时间窗等约束全部满足。

3.车辆使用方案分析

根据求解结果，本次调度方案共启用车辆 73 辆，其中各车型使用数量如下：燃油车 A：33 辆，燃油车 B：31 辆，燃油车 C：1 辆，新能源车 A：8 辆。客户订单的重量分布（同一客户不同订单重量合并）如图 7 所示。整体呈现“小订单多、大订单少”的特征：近半数客户订单重量在 2000kg 以下，同时少量订单重量超过10000kg，对车型的载重能力提出了不同要求。

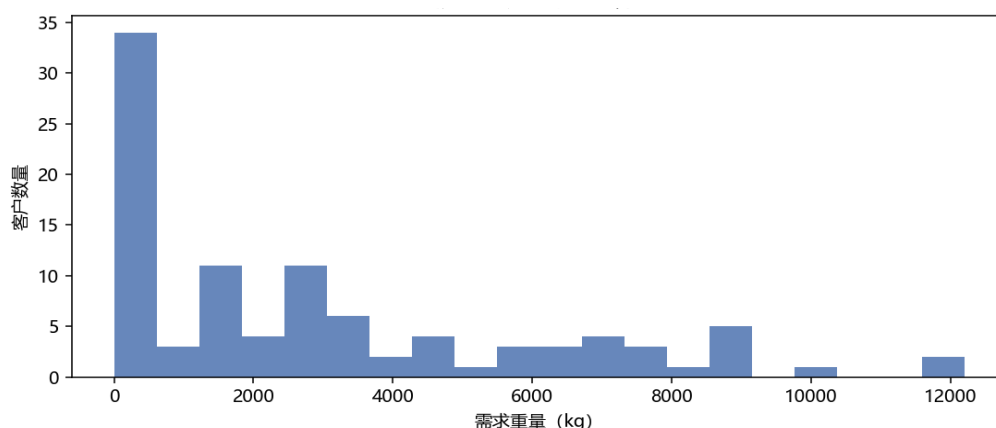


图 7 客户需求重量分布

从车型使用与订单匹配来看，本次调度方案中车辆配置与订单需求结构整体契合度较高。由图 7 可以看出，订单以中小批量为主、大额订单为辅，因此在车型安排上，燃油车 A、B 依托数量充足、运营成本较低的特点，承担了大部分中小批量常规配送任务，构成车队的主体运力；新能源车 A 则主要对应高重量区间的大额订单，发挥其载重和容积方面的优势。各车型载重利用率均较高，车辆闲置现象较少，资源浪费得到有效控制，车队配置在成本与效率之间实现了较为合理的平衡，运力资源利用较为充分。

4.行驶路径规划分析

本次调度共形成 121 条配送路线，由于路线方案过多，本文仅呈现部分配送路线，如表 5 所示。

表 5 配送路线明细

线路编号	开车时刻/分钟	车辆类型	配送路线	服务客户数
5	600	燃油车 A	0->60->7->73->92->90->19->42->55->61->75->0	10
27	750	燃油车 A	0->48->46->87->91->0	4
52	475	燃油车 A	0->42->0	1
61	950	新能源车 A	0->43->34->10->0	3
70	1025	新能源车 A	0->53->0	1
104	555	燃油车 B	0->85->79->93->0	3
113	610	燃油车 B	0->82->26->0	2
121	660	燃油车 C	0->13->29->0	2

每条路线均为“从配送中心出发、服务客户后返回配送中心”的闭合路径，例如路线 5 的路径为：0→60→7→73→92→90→19→42→55→61→75→0，单次服务客户 10 个，是本次方案中服务客户数最多的路线。

从整体路径特征来看，总行驶里程为 16748.76 公里，单车平均行驶里程与路线平均服务客户数均处于较优区间；路线服务客户数量分布较为合理，多数路线单次服务1–3 个客户，少部分路线服务 5–10 个客户，在提升路径效率的同时兼顾了客户服务密度。总体而言，各条路线均符合车辆调度的基本运营规范，发车时间主要集中在白天时段，与客户收货时间需求匹配度较高，路径规划的合理性与可行性得到了有效保障。

5.成本与碳排放构成分析

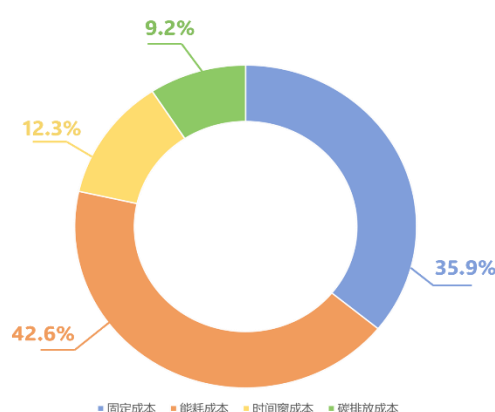


图 8 总成本构成

本次调度方案的总成本为 85455.83 元，各项成本的占比结构如图 8 所示。其中能耗成本占比最高，达 42.6%，是方案成本的主要来源；固定成本占比 35.9%，位列第二；时间窗成本与碳成本占比相对较低，分别为 12.3% 和 9.2%。这一结构与方案的运营特征

高度契合：能耗成本占主导，符合物流配送成本的一般规律；固定成本占比次之，说明车队规模已得到有效控制；时间窗成本占比不高，表明发车时刻优化效果显著；碳成本占比约一成，兼顾了低碳配送的要求。整体来看，成本结构分布合理，未出现异常偏高的成本项，验证了方案的优化效果与可行性。

6.2 问题二模型建立与求解

6.2.1 问题二模型建立

在问题一所建立的静态多车型 VRPTW 优化模型基础上，进一步将城市绿色配送区的限行政策纳入考虑，使模型更贴近实际物流运行环境。根据政策规定，每日 8:00–16:00 期间，燃油车辆不得进入以市中心为圆心、半径 10 km 的绿色配送区。该约束会对车辆路径安排、服务时间以及车型选择产生直接影响，使原有调度方案难以继续适用。

因此，在原模型框架下，本文补充引入限行合规性约束，并采用字典序的多级优化目标，优先保证调度方案满足政策要求，在此基础上再兼顾运输效率与成本。由此建立了基于绿色配送区限行政策下的多车型 VRPTW 优化模型，为限行背景下的城市物流调度提供更具可行性的决策依据。

因问题二为问题一模型的拓展与优化，基础符号体系、目标函数与通用约束与前文基本保持一致。为避免重复，本文仅针对绿色配送区限行政策补充专属约束与目标函数调整，不再赘述已定义的基础内容。

1. 目标函数：

为保证政策合规的绝对优先性，问题二采用字典序多级目标^[4]，通过大额违规惩罚系数实现线性化求解，其目标函数表达式如下：

$$\min C = \sum_{i=1}^4 C_i + O \cdot \sum_{r \in R_k} \sum_{i \in N} \phi_{ri} \quad (23)$$

其中 N 为绿色配送区内客户集合； $O \gg \sum_{i=1}^4 C_i$ ，确保合规解的成本显著低于违规解，实现字典序优先； $\phi_{ri} \in \{0,1\}$ 为路线 r 服务客户 i 的违规指示变量。

特别的，当且仅当以下三个条件同时满足时， $\phi_{ri} = 1$ ，否则为 0：

- (1) 路线 r 采用燃油车型；
- (2) 客户 i 位于绿色配送区内；
- (3) 路线 r 到达客户 i 的时刻 t_{ik}^r 处于限时时段，即 $t_{ik}^r \in [480, 960)$ 。

2.约束条件:

(1) **燃油车限行时段合规性约束:** 若路线 r 使用燃油车服务区内客户 i , 则到达时刻 t_{ik}^r 必须 ≤ 480 或 ≥ 960 , 即不再限时时段内。其公式如下:

$$\begin{cases} t_{ik}^r \leq 480 + D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N \\ t_{ik}^r \geq 960 - D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N \end{cases} \quad (24)$$

其中 $m_r^f \in \{0,1\}$ 表示路线 r 是否选用燃油车型, 取 1 表示燃油车, 0 表示新能源车;

$g_i \in \{0,1\}$ 表示客户 i 是否位于绿色配送区内, 取 1 表示在区内, 0 表示在区外;

D 取一个足够大的正数, 本文取 D 为 1000000。

(2) **区内客户服务权限约束:** 在限行时段内, 绿色配送区内的客户仅可由新能源车服务。其公式如下:

$$z_{ri} \cdot m_r^f \leq 1 - \frac{t_{ik}^r - 480}{D} - \frac{960 - t_{ik}^r}{D}, \quad \forall r \in R_k, i \in N' \quad (25)$$

(3) **跨区路线时序衔接约束:** 对于“配送中心到区内客户再到区外客户”的路线, 若使用燃油车, 需保证区内路段的行驶与服务均在非限行时段完成。其公式如下:

$$\begin{cases} t_r^{start} + \sum_{(p,q) \text{ 在路线 } r \text{ 中位于 } i \text{ 之前}} \frac{d_{pq}}{v(t_{pq}^{start})} \cdot 60 \leq 480 + D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N' \\ t_r^{start} + \sum_{(p,q) \text{ 在路线 } r \text{ 中位于 } i \text{ 之前}} \frac{d_{pq}}{v(t_{pq}^{start})} \cdot 60 \geq 960 - D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N' \end{cases} \quad (26)$$

其中 t_r^{start} 为配送线路 r 的计划发车时刻; $v(t_{pq}^{start})$ 为路段 (p,q) 出发时刻对应的行驶速度。

综上可得, 基于绿色配送区限行政策下的多车型 VRPTW 优化模型构建如下:

$$\begin{aligned}
\min C &= \sum_{i=1}^4 C_i + O \cdot \sum_{r \in R_k} \sum_{i \in N'} \phi_{ri} \\
\left\{ \begin{array}{ll}
\sum_{r \in R_k} q_{ri} = q_i', \sum_{r \in R_k} u_{ri} = u_i', \sum_{r \in R_k} z_{ri} = 1 & \forall i \in N \\
\sum_{i \in N} q_{ri} \leq Q_{kr}, \sum_{i \in N} u_{ri} \leq V_{kr}, & \forall r \in R_k \\
\sum_{j \in V'} x_{ijk}^r = \sum_{j \in V} x_{jik}^r, & \forall i \in N, r \in R_k \\
\sum_{j \in N} x_{0jk}^r = \sum_{j \in N} x_{j0k}^r = 1, \sum_{k \in K} m_k^r = 1, & \forall r \in R_k \\
P_k' \leq P_k, & \forall k \in K \\
t_{ik}^r \leq 480 + D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N \\
t_{ik}^r \geq 960 - D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N \\
z_{ri} \cdot m_r^f \leq 1 - \frac{t_{ik}^r - 480}{D} - \frac{960 - t_{ik}^r}{D}, & \forall r \in R_k, i \in N' \\
t_r^{start} + \sum_{(p,q) \text{ 在路线 } r \text{ 中位于 } i \text{ 之前}} \frac{d_{pq}}{v(t_{pq}^{start})} \cdot 60 \leq 480 + D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N' \\
t_r^{start} + \sum_{(p,q) \text{ 在路线 } r \text{ 中位于 } i \text{ 之前}} \frac{d_{pq}}{v(t_{pq}^{start})} \cdot 60 \geq 960 - D \cdot (1 - z_{ri} \cdot m_r^f \cdot g_i), & \forall r \in R_k, i \in N' \\
q_{ri}, u_{ri}, t_{ik}, t_{ik}' \geq 0 \\
x_{ijk}^r, m_k^r \in \{0, 1\}
\end{array} \right. \quad (27)
\end{aligned}$$

问题二在问题一基础上引入限行政策，采用线性化约束与字典序目标，优先保障政策合规，再实现成本与效率的协同优化。

6.2.2 问题二模型求解与分析

本问在问题一多车型车辆路径问题的基础上，引入绿色配送区限行政策，新增“空间—时间—车型”三重耦合约束，直接改变了配送路径的时间排布逻辑与车型分配策略。基于改进后的模型与算法，本文求解得到完全合规的调度方案，其核心结果及与问题一的对比情况如下表 6 所示。

表 6 问题一与问题二核心结果对比

指标项	问题一（无限行约束）	问题二（绿色限行约束）	变化幅度
总成本/元	85455.83	94849.17	+10.99%
总行驶里程/公里	16748.76	16903.75	+0.93%
启用车辆总数/辆	73	83	+13.70%
燃油车数量/辆	65	76	+16.92%
新能源车数量/辆	8	7	-12.5%
形成配送路线/条	121	121	0
碳排放总量/千克	12237	12694	+3.7%

由表 6 可知，绿色配送区限行政策对车队配置与运营成本均产生了结构性影响，方案总成本由 85455.83 元增至 94849.17 元，涨幅 10.99%；总行驶里程基本持平，仅小幅上升 0.93%；为满足管控区配送需求，启用车辆总数由 73 辆增至 83 辆，增幅 13.70%，其中燃油车数量由 65 辆增至 76 辆，新能源车数量由 8 辆微调至 7 辆，路线总数保持 121 条不变；碳排放总量从 12237kg 增加到 12694kg，增幅整体可控。

为更加直观呈现绿色配送区政策对调度方案的影响，本文将问题二的求解结果与问题一进行多维度对比分析。

1. 路径方案与合规性验证

本文为直观呈现限行政策对配送路径分布的影响，通过配送路线方案示意图，对比问题一与问题二的路径。其示意如图 9 所示。

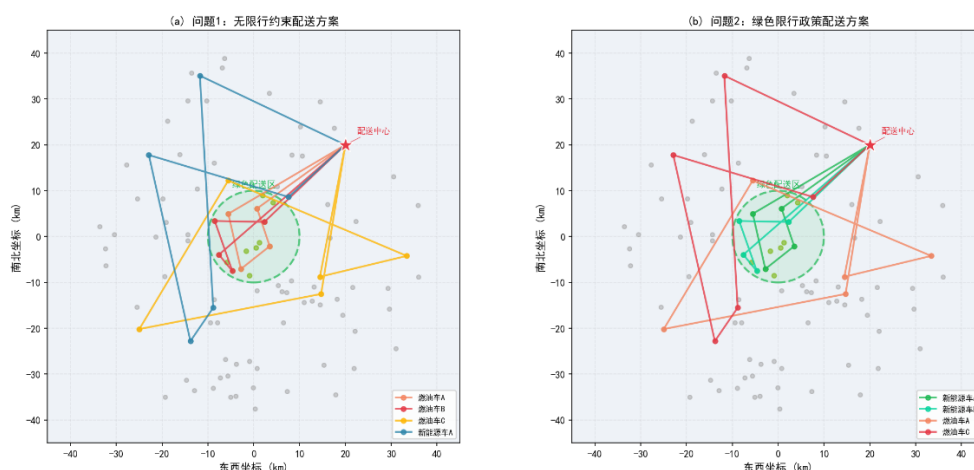


图 9 配送路线方案对比示意图

从图 9 (a) 可以看到，问题一方案中，燃油车路线可自由进入绿色配送区，不同车型路线在管控区内无明显分区；而在 (b) 的问题二方案中，管控区内的客户点已全部由新能源车路线覆盖，燃油车路线均绕行避开了绿色配送区，在管控区外围服务。这种路径分布的显著差异，直接体现了限行政策的约束效果。

本文进一步对方案进行合规性校验，结果表明：问题二方案中，所有绿色配送区内的客户点，在 8:00-16:00 时段内均由新能源车提供服务，燃油车未出现违规进入的情况；所有燃油车路线中，绿色区客户点的到达时间均晚于 16:00，完全满足限行政策要求；客户服务时间窗合规率保持在较高水平，未出现严重超时现象，方案的可行性与合规性得到了充分保障。

2. 车辆使用结构与成本构成的综合变化分析

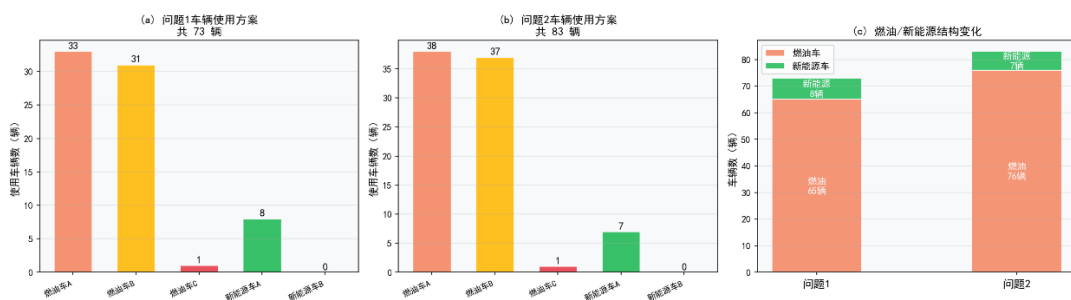


图 10 问题一与问题二车辆使用结构对比图

结合图 10 可以看出，限行政策使车队分工更加清晰：燃油车 A、B 的使用数量有所增加，主要承担管控区外订单以及 16:00 限行解除后的城区补送任务；新能源车 A 数量虽略有下降，但配送任务进一步集中于绿色配送区内，车辆利用率反而提升。整体来看，路线总数基本保持稳定，通过调整车型配比与配送时段，逐步形成“燃油车服务外围、新能源车保障城区”的运行模式，在满足政策约束的同时，将成本控制在可接受范围内，体现出较好的运营协调性。进一步从成本结构分析，限行政策对车队配置与成本构成均产生了明显影响：燃油车承担更多外围任务，新能源车聚焦核心区域，分工差异更加显著；受新能源车占比提升及配送时段调整影响，总成本较问题一上升约 10.99%，但整体仍处于可接受区间，其中固定成本与时间窗相关成本增长较为明显，而碳排放成本增幅相对较小，表明在兼顾环保约束的同时，方案在经济性与可行性之间实现了较好的平衡。

3.碳排放与减排效益综合分析

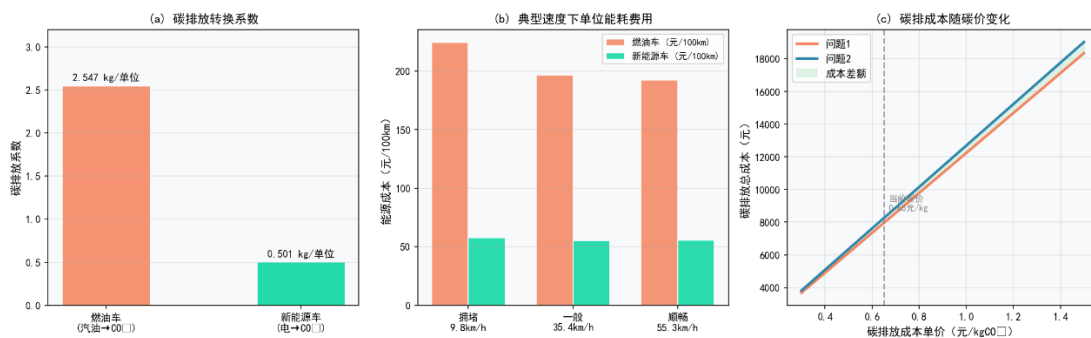


图 11 碳排放成本与减排效益分析图

综合对比问题一与问题二的求解结果可以看出，绿色配送区限行政策对车辆调度方案产生了多方面影响。如图 11 (a) 所示，新能源车辆的碳排放系数显著低于燃油车辆，说明其在单位运输过程中的碳排放更低，是实现减排目标的关键因素；同时，由图 11 (b) 可见，在不同运行速度下，新能源车辆的单位能耗成本整体低于燃油车辆，体现出更优的能源利用效率；进一步结合图 11 (c)，在碳价逐步上升的情景下，实施限行政策后的调度方案在碳成本增长趋势上相较问题一有所缓解，表明绿色调度策略在控制碳成本方面具有积极作用。

基于上述分析，在环境效益方面，碳排放总量得到有效控制，绿色配送目标基本实现；在运营成本方面，尽管新能源车辆占比提升及部分订单服务时段调整使总成本略有增加，但整体仍处于可接受范围；同时，在车队结构方面，新能源车辆的使用比例与利用率明显提高，推动车队向低碳化方向优化，为后续在类似政策约束下开展车辆调度提供了实践依据。

6.3 问题三模型建立与求解

6.3.1 问题三模型建立

在运营过程中，订单执行常面临各类不确定扰动，原有静态预优化方案将不再适配现实场景。为提升配送系统的抗干扰能力与动态调整水平，本文针对订单取消、新增订单、配送地址变更、客户时间窗调整以及组合事件等高频突发场景，构建事件触发式局部重调模型，在保障原有载重与容积约束、车速时变约束以及限行政策约束不变的前提下，实现毫秒级快速响应与全局配送成本最优。

在事件触发时刻 t_{event} 将所有路线划分为两层，分别为冻结层和激活层。冻结层是指已完成或正在完成的服务，该类服务不可更改；激活层是指未开始的服务，可以进行路线重规划。接下来针对四类突发事件及组合事件的应对策略分别建立模型进行处理。

6.3.1.1 订单取消事件处理

当配送途中发生订单取消事件^[5]时，首先判定操作有效性：若客户已在事件发生时刻前完成配送，则本次取消操作无效，不改动现有配送方案；若客户仍处于未配送状态，则执行动态调整。

遍历所有路线并移除目标客户剩余访问点位，其公式如下：

$$\forall r \in R, \quad Y'_r = Y_r \setminus \{i\} \quad (28)$$

其中 Y_r 为路线 r 剩余待访问客户节点集合， y 为待取消的目标订单客户节点。若更新后线路 r 无剩余待服务节点，即 $Y'_r = \emptyset$ ，则直接撤销该路线，节省车辆固定成本。

对所有受影响且尚未开始的节点，重新迭代求解最优发车时刻，以满足剩余节点时间窗与时变车速等约束。同时，对于订单取消带来的总成本变化量为

$$\Delta C_{cancel} = -F_{k,r} \cdot \mathbb{I}\{Y'_r = \emptyset\} - \Delta C_{2,r} - \Delta C_{3,r} - \Delta C_{4,i} \quad (29)$$

其中 $\mathbb{I}\{Y'_r = \emptyset\}$ 为指示函数。

6.3.1.2 新增订单事件处理

在实时配送调度场景中，临时新增客户订单与追加需求会打破原有路线的容量平衡与路径最优状态。因此，本文采用最小增量插入式算法，先校验车辆载重、容积可行性，

再全局搜寻最优路线与插入位置，实现低成本就近插单。当无可用路线时则优选派最低成本车型新开线路，保障配送系统稳定经济运行。

对每条激活路线 r 校验插入新增订单后的载重与容积是否满足约束：

$$\sum_{i \in Y_r} q_{ri} + \bar{q} \leq Q_{rk} \wedge \sum_{i \in Y_r} u_{ri} + \bar{u} \leq V_{rk} \quad (30)$$

其中 $\bar{q}$ 为新增订单货物的重量， $\bar{u}$ 为新增订单货物的体积。

若满足载重与容积约束，则计算新节点 i' 插入第 l 节点带来的距离增量，即移除路段 $i_{l-1} \rightarrow i_l$ ，新增绕行路段 $i_{l-1} \rightarrow i' \rightarrow i_l$ ：

$$\Delta d(r, i', l) = d_{i_{l-1}, i'} + d_{i', i_l} - d_{i_{l-1}, i_l} \quad (31)$$

遍历所有可行路线、插入位置，选取增量成本最小的插入方案：

$$(r^*, l^*) = \arg \min_{(r, l) \in \Omega} \Delta d(r, i', l) \quad (32)$$

其中 $\Omega = \left\{ (r, l) \left| \sum_{i \in Y_r} q_{ri} + \bar{q} \leq Q_{rk} \wedge \sum_{i \in Y_r} u_{ri} + \bar{u} \leq V_{rk} \right. \right\}$ 。最优位置插入完成后，对该线路节点进行最近邻排序，并对尚未发车的路线重新迭代求解最优发车方案。

当全部现有活跃路线均无法容纳新增订单需求时，则选取其他车辆组建全新配送路线，车型选择模型如下：

$$k' = \arg \min_{k: Q_k \geq \bar{q}, V_k \geq \bar{u}, P_k^{used} < P_k} F_k \quad (33)$$

即在满足载重容积要求且仍有可用车辆配额的前提下，选择固定启用成本最低的车辆，以期最大程度控制新增运营成本。

6.3.1.3 配送地址变更事件处理

客户配送位置变更后，原有路网距离关系发生改变，初始预设的最优配送路径不再具备较高的经济效应与合理性，若仍沿用原路线执行，将产生大量无效绕行与额外成本。因此本文根据两类场景分别构建地址变更动态调整模型，在保障车辆载重、容积等基础约束不变的前提下，迅速重新规划全局最优配送方案，实现扰动下的成本最优适配。

1. 场景一：订单已送达客户旧地址

此时旧地址配送服务已完成且不可撤销，因此，可将此类事件等效为新增订单问题。

2. 场景二：该订单尚未配送

地址变更相当于先取消旧地址订单再新增新地址订单的组合操作，其处理过程如下：

Step1: 从所有激活路线的待配送集合中移除客户节点，移除产生的距离成本 $\Delta d^- \leq 0$ ；

Step2: 更新全局距离矩阵，得到新距离矩阵 d' ；

Step3: 基于新距离矩阵，对更新后的客户节点 i'' 执行插入线路 r 最优决策：

$$(r^*, l^*) = \arg \min_{(r, l) \in \Omega} \Delta d'(r, i'', l) \quad (34)$$

该类场景总距离成本变化由移除旧路段节省的距离成本与插入新路段增加的距离成本之差所决定：

$$\Delta d_{address} = \Delta d^-(\cdot) + \Delta d^+(r^*, i'', l^*) \quad (35)$$

6.3.1.4 时间窗调整事件处理

在配送执行过程中，客户临时更改可收货时间，是典型的时间维度扰动^[6]。客户时间窗收紧或放宽后，原有路线的到达时序、等待与迟到惩罚成本会发生显著变化，原本最优的发车策略与成本平衡状态被打破。为快速响应该扰动、兼顾配送可行性与整体经济性，本节构建时间窗调整动态适配数学模型，通过更新时间约束、重估发车时刻、重新核算时间惩罚成本的方式，完成低成本高效修复。

记客户 i 的时间窗口 $[a_i, b_i]$ 更新为 $[a'_i, b'_i]$ 。对所有尚未发车的受影响线路，基于新的时间窗口约束重新求解最优发车方案。时间窗调整后，该客户的时间成本变化量为：

$$\Delta C_{4,i} = C^+ \cdot \max\left(\left\lceil \frac{a'_i - t_i}{60} \right\rceil + 1, 0\right) + C^- \cdot \max\left(\left\lceil \frac{t_i - b'_i}{60} \right\rceil + 1, 0\right) - C_{4,i}^0 \quad (36)$$

其中 $C_{4,i}^0$ 为时间窗口调整前客户 i 原有软时间窗口总成本。

时间窗调整分为时间窗收紧与时间窗放宽两类典型情形，二者对现有配送方案的影响程度与调度调整策略存在显著差异：

1. 时间窗口收紧

客户可收货时间区间压缩，原本合规的到达时序可能产生额外早等待或迟到惩罚。模型通过重优化发车时刻与调整节点访问顺序，重新平衡时间成本，规避高额违约损失。

2. 时间窗口放宽

客户可收货时间区间扩大，原有到达方案的时间违规风险降低，整体时间惩罚成本大概率下降，仅需更新参数与成本核算，无需大规模重新规划路线。

6.3.1.5 组合事件处理优先级数学模型

当订单取消、地址变更、时间窗口调整、新增订单多类动态扰动同时触发时，需按照全局优先顺序执行修复算子^[7]，保证容量资源最大化利用、结构优先于时间优化。

定义四类扰动事件集合，如下表 7 所示：

表 7 扰动事件集合

符号	事件集合
L_1	订单取消
L_2	新增订单
L_3	配送地址变更
L_4	时间窗调整

根据订单配送的实际情况，我们做出如下调整原则：

订单取消 $\xrightarrow{\text{释放容量}}$ 配送地址变更 $\xrightarrow{\text{更新约束}}$ 时间窗调整 $\xrightarrow{\text{重新安排发车}}$ 新增订单 (37)

- (1) 订单取消操作释放载重与体积容量，为后续新增订单创造更多可用插入路线；
- (2) 地址变更优先于时间窗调整，确保路线结构稳定后再优化时间参数；
- (3) 新增订单最后执行，充分利用前面步骤清理出的插入空间。

根据调整原则定义事件优先级权重：

$$\sigma(L_1) > \sigma(L_2) > \sigma(L_3) > \sigma(L_4) \quad (38)$$

当多事件同时触发时，全局执行顺序满足：

$$\arg \max_{L \in \{L_1, L_2, L_3, L_4\}} \sigma(L) \quad (39)$$

即始终优先选取当前剩余集合中优先级最的事件进行调整，直至扰动全部处理完毕。

6.3.2 问题三模型求解与分析

本文基于问题二的调度模型，选取9:30作为扰动触发时刻，对四类独立扰动与多事件组合并发扰动开展仿真实验。通过测算扰动前后配送综合成本、资源利用率等关键指标的变化，量化评估不同扰动类型对配送系统的实际影响。由于文章篇幅有限，部分结果表不作展示。

6.3.2.1 订单取消事件处理结果

假设客户 49 于9:30取消订单，将该客户节点从路线[26,64]中删除后，检测发现路线[26]无剩余节点，因此，可以直接撤销该路线。对路线[64]尚未开始的配送节点进行重规划，最终结果如表 8 所示：

表 8 客户 49 订单取消处理结果

指标	事件前	事件后	变化量
总成本(元)	94849.17	94723.53	-125.64
固定成本	33200.00	33200.00	0.00
能耗成本	37994.21	37818.98	-175.23
碳排放成本	8251.33	8213.20	-38.12
软时间窗成本	15403.63	15491.35	+87.72

碳排放总量	12694.35	12635.70	-58.65
使用车辆数	121	120	-1
政策违规次数	0	0	0
响应时间(ms)		10.7	

6.3.2.2 新增订单事件处理结果

假设客户 1 于 9:30 新增紧急订单，订单货物的重量为 500kg，体积为 $1.2m^3$ ，可收货时间为 11:33–12:22。经检测可知所有激活路线均无法容纳新增订单，因此新开燃油车为其配送货物。订单处理结果如表 9 所示：

表 9 客户 1 新增订单处理结果

指标	事件前	事件后	变化量
总成本(元)	94849.17	95649.82	+800.65
固定成本	33200.00	33600.00	+400.00
能耗成本	37994.21	38143.46	+149.25
碳排放成本	8251.33	8283.79	+32.47
软时间窗成本	15403.63	15622.57	+218.94
碳排放总量	12694.35	12744.30	+49.95
使用车辆数	121	122	+1
政策违规次数	0	0	0
响应时间(ms)		11.2	

6.3.2.3 配送地址变更事件处理结果

假设客户 57 于 9:30 将货物配送地址变更为 (12.0, -4.0)，需重新规划路径位置。将该客户节点从路线 [13, 47, 49] 删除后，检测得线路 [13, 47] 已无剩余节点，因此新派两辆燃油车分别配送 3000kg 货物。另外，将节点重新插入路线 [49]，此时距离增量约为 3.6km。订单处理结果不作展示。

6.3.2.4 时间窗调整事件处理结果

假设客户 16 于 9:30 申请时间窗延长至 24:00，即由 [9:17, 10:41] 变为 [9:17, 24:00]，而在问题二的求解结果中配送车辆实际送达时间为 22:55，因此，重新规格发车时刻：路线 [42] 发车时刻保持不变，仍为 17:05，路线 [114] 发车时刻调整为 18:30。订单处理结果不作展示。

6.3.2.5 组合事件处理结果

假设客户 55 于 9:30 取消订单，客户 1 此时新增订单（新增货物重量为 300kg，体积为 $0.8m^3$ ）。根据事件优先级，我们先处理订单取消事件。将客户 55 从路线 [6, 8, 50, 55, 65]

中删除，检测得路线[8,50,55,65]已空，接着对路线[6]进行重排。对于客户1，其所有活跃路线均无法容纳新增订单，因此新派燃油车进行配送。订单处理结果如表 10 所示：

表 10 组合事件处理结果

指标	事件前	事件后	变化量
总成本(元)	94849.17	93026.76	-1822.41
固定成本	33200.00	32400.00	-800.00
能耗成本	37994.21	37040.66	-953.55
碳排放成本	8251.33	8046.99	-204.34
软时间窗成本	15403.63	15539.11	+135.48
碳排放总量	12694.35	12379.98	-314.37
使用车辆数	121	118	-3
政策违规次数	0	0	0
响应时间(ms)	9.8		

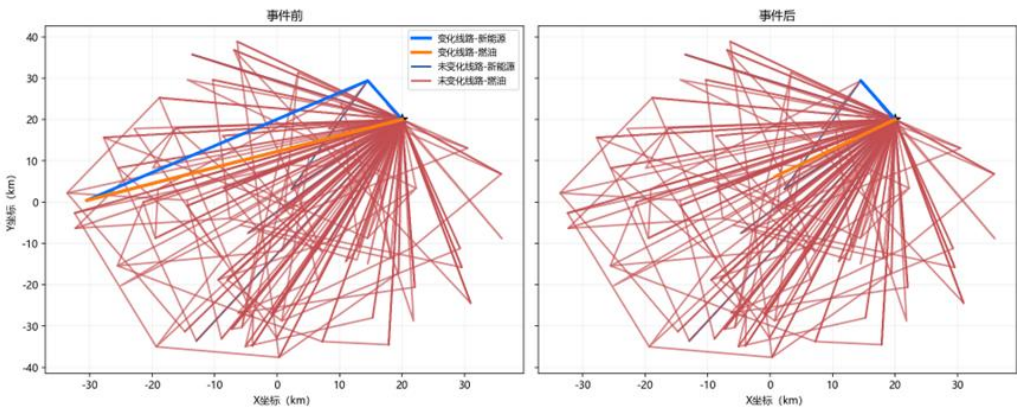


图 12 组合事件发生前后路线对比图

图 12 为该组合扰动发生前后路线拓扑对比，绝大多数原有燃油与新能源配送路线保持不变，仅少量受扰动影响的线路完成局部路径微调。该组合扰动调度模型在快速适配供需变动、完成动态重调度的同时，避免了大范围路径重构。不仅降低了重调度的运算冗余，也缩短了响应时耗，同时也能够有效压缩整体行驶里程与综合运营成本，体现了其极强的稳定性、适配性与路径优化能力。

6.3.2.6 不同扰动事件处理效果对比

1.不同扰动场景算法响应时间分析

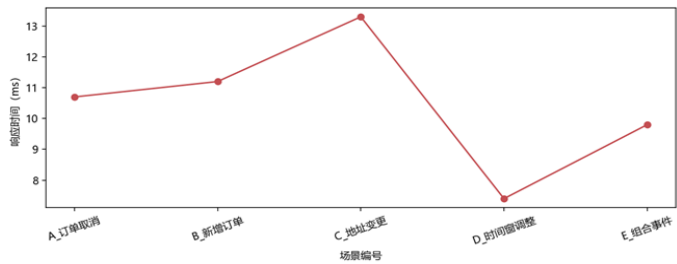


图 13 响应时间

如图 13，五种扰动场景下模型响应时间均维持在毫秒级，其中地址变更运算复杂度最高，响应耗时最长；时间窗调整仅需优化时间参数，响应效率最优；取消、新增订单与组合事件的响应速度均较快。整体算法调度响应迅速，适配订单配送的即时性需求。

2.总成本优化效果分析

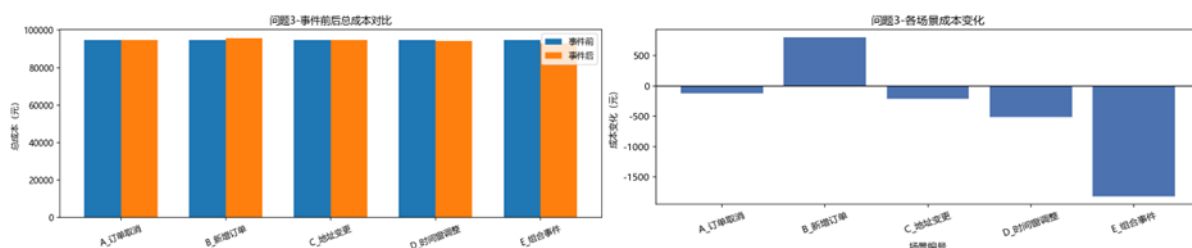


图 14 不同扰动场景下调度前后总成本及成本变化对比

由图 14 可得，除新增订单场景因业务量增加导致总成本小幅度上升外，其余所有扰动场景经过模型动态调度调整后，总成本均实现不同幅度下降。其中组合场景下成本优化幅度最为显著，降低成本优势突出。整体来看，该多扰动协同处理模型能够在各类突发状况下有效控制运营支出，具备优异的经济性与成本优化能力。

七、灵敏度分析

7.1 基于单参数扰动法的灵敏度分析

单参数扰动法是指每次变动一个目标参数，其余参数保持不变，重新求解后记录核心指标的响应变化。本文选取车速期望值参数 μ 作为参数进行影响分析。

速度时变模型中各时段车速服从正态分布，均值受实际交通条件影响。将三个时段的速度期望值同比例缩放，取扰动倍数 $\mu \in \{0.8, 0.85, 0.9, 1.0, 1.1, 1.2\}$ ，对比分析总成本与能耗成本的响应规律，结果如图 15 所示。

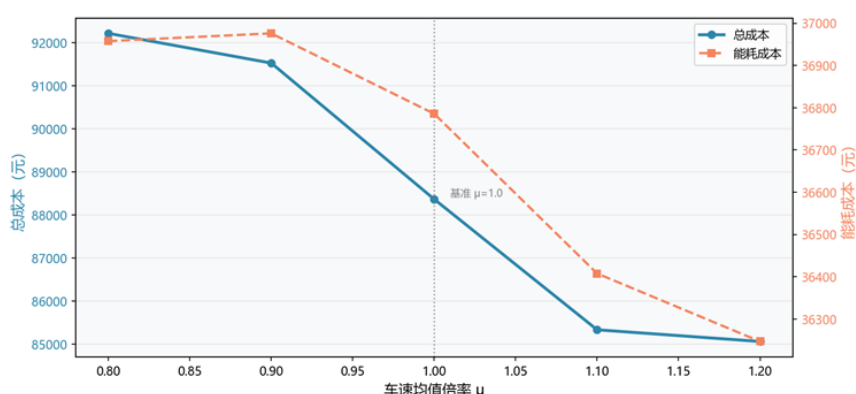


图 15 车速均值倍率灵敏度分析

由图 15 可知，总成本与能耗成本均随速度期望值的提升单调下降。当 μ 从 0.80 增大至 1.20 时，总成本由约 92,200 元降至约 85,000 元，降幅约为 7.8%；能耗成本由约

37000 元降至约 36,200 元,降幅相对平缓。以基准值 ($\mu=1.0$) 为参照,速度降低 10%,即 $\mu=0.9$ 时总成本上升约 3.5%,而速度提升 10%,即 $\mu=1.1$ 时总成本下降约 3.5%,两侧响应基本对称。上述规律与能耗 U 型曲线在低速区间的特征基本吻合,车速过低时行程时间延长,时间窗惩罚与能耗均升高,共同推高了总成本。在 $\mu \in [0.9,1.1]$ 的合理扰动范围内,总成本波动幅度不超过 4%,表明模型对一般性交通波动具备较强的鲁棒性。

7.2 基于政策情景枚举的灵敏度分析

政策情景枚举法针对外生政策参数,通过系统枚举各离散取值水平下的最优解,识别政策边界变化的成本拐点,为政策设计提供量化依据。本文对绿色配送区半径 R 进行重点分析。

取步长为 1km ,将绿区半径在 5km 至 20km 之间逐一枚举,记录每组方案的总成本与新能源运力缺口比例 $\Delta Q(R)$,结果如图 16 所示。

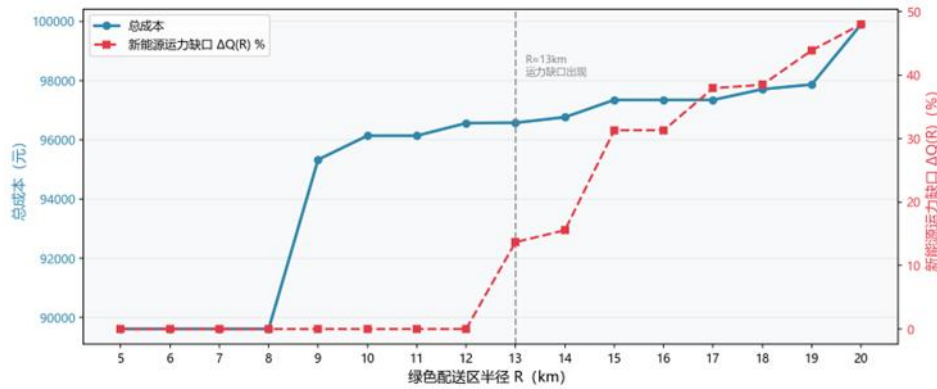


图 16 绿色配送区半径灵敏度分析

当半径从 8km 扩至 9km 时,总成本从约 89,800 元骤升至约 95,400 元,增幅约 6.2%,其原因可能是更多客户点被纳入限行范围,新能源车辆调度压力突然增大,迫使模型大幅调整路线结构。当半径超过 12km 时,新能源运力缺口 $\Delta Q(R)$ 由 0 急剧攀升至约 10% 以上,此后随半径扩大持续增长,至 $R=20\text{km}$ 时达到约 48%。与此同时,总成本进一步上升至约 100,000 元,表明在 $R=13\text{km}$ 处,绿色配送区内的业务需求已超出现有新能源车队的运力上限,模型被迫采用绕路或分次配送等高成本补偿策略。

7.3 基于蒙特卡洛模拟^[8]的随机鲁棒性验证

速度时变模型中各时段车速服从正态分布,参数估计本身存在不确定性,导致单次求解结果无法清晰刻画随机环境下的方案。为此,本文对速度分布参数进行 $N=300$ 次独立随机抽样,分别在问题一与问题二条件下重复求解,统计总成本的概率分布。

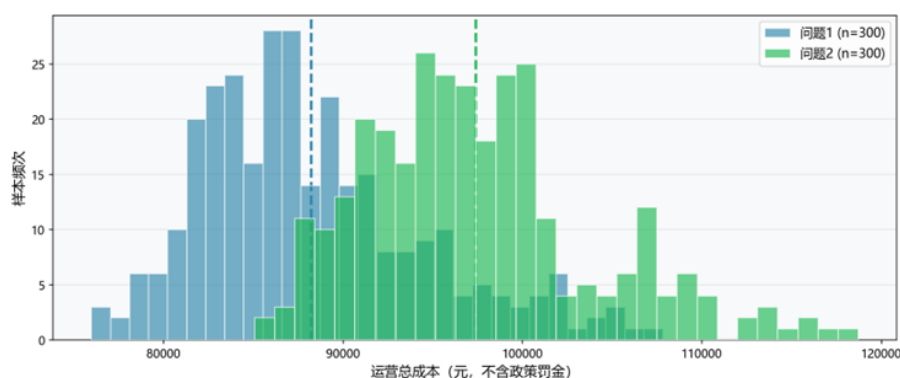


图 17 蒙特卡洛总成本分布

问题一方案的总成本分布集中且对称，均值约为 88500 元，分布范围约为 78,000–93,000 元，区间宽度约 15,000 元，分布形态接近单峰正态，说明无限制条件下的方案受随机交通波动的影响较为有限，整体鲁棒性良好。

问题二方案的总成本分布整体右移，均值约为 97,000 元，相比问题一均值上升约 8,500 元，涨幅约 9.6%，分布范围扩展至 83,000–120,000 元，区间宽度超过 37,000 元，尾部出现明显的右偏拖尾，表明绿色配送区限行政策在抬高平均运营成本的同时，也显著放大了方案对交通随机性的敏感程度。当交通条件变差时，新能源车辆调度余量不足，成本可能出现较大幅度的超支。

八、模型评价与推广

8.1 模型的优点

（1）本文在模型中创新引入载重率修正系数，精准刻画车辆行驶能耗与实时载重之间的动态关联关系，还原出不同载重状态下的能耗水平，让成本测算与路径规划结果更贴合实际运行规律，有效提升了模型计算精度与现实贴合度。

（2）问题二构建了大额惩罚系数的线性化字典序目标，通过设置大额惩罚系数，将政策合规性设为第一优先级保障目标，在此基础上再统筹优化配送运营成本与碳排放水平，该目标设计兼顾政策刚性与企业运营目标。

（3）问题三的动态调度模型具备优异的突发扰动适配能力。面对单类及多类叠加突发事件，模型可在保留原有配送主体框架的基础上完成局部快速重调度，既维持了配送系统整体稳定，又能有效控制扰动带来的成本波动。

8.2 模型的不足

（1）本文模型对订单的拆分与合并处理仅采用理想化规则，未考虑不同订单的货物属性、装卸限制、客户要求等差异化约束。当前简化的订单拆分与真实业务的精细化运营需求仍存在一定差距，后续可引入多维度约束的智能订单拆分算法进一步优化。

(2) 在组合事件协同调度模型中，本文仅选取了四类典型的扰动事件开展仿真验证，而实际配送运营过程中，还普遍存在极端天气、道路临时管控、车辆中途故障、大范围交通拥堵等更多不可预见的复杂突发状况。本文的模型目前对极端复合、未知随机扰动的应对能力有一定限度。

8.3 模型的改进与推广

8.3.1 模型的改进

在“双碳”战略背景下，碳排放成本已成为物流配送调度不可忽视的重要因素。为贴合市场化低碳管控规则，提升模型的实用价值，本文引入碳配额与碳交易机制^[9]，构建兼顾运营效率与绿色减排的优化调度模型。

当配送方案实际产生的总碳排放量 $W \leq$ 政府下发的初始碳排放配额 W_0 时，不需要支付额外的碳交易成本；否则需根据超额碳排放量购入碳排放配额，从而产生碳额外支出。该模型产生的碳交易成本 C_{carbon} 为

$$C_{carbon} = \begin{cases} 0, & W \leq W_0 \\ \varphi_c \cdot (W - W_0), & W > W_0 \end{cases} \quad (40)$$

其中 φ_c 为碳交易市场单位碳排放权交易单价。

8.3.2 模型的推广

本文所构建的多扰动动态协同调度优化模型，不仅能够很好适配城市同城即时配送的复杂场景，也可推广至各类动态车辆路径与资源调配问题。在民生物流领域，可应用于生鲜冷链配送^[10]，兼顾时效保障与动态订单变动调整；在公共服务领域，可支撑城市医疗急救、便民物资的动态应急调度，实现突发需求下的运力最优匹配；在工业生产领域，可服务于工业园区厂区物料的周转运输，适配生产计划临时变更的调度需求。总体来看，该方法对多约束、强动态、多变化的调度决策问题具备较强的通用性与迁移推广价值。

参考文献

- [1] 曹庆奎,张茜茜,任向阳.基于需求可拆分的多车型应急物资配送路径优化研究[J].邢台学院学报,2023,38(03):178-186.
- [2] 曹庆奎,杨凯文,任向阳,等.基于交通流的多模糊时间窗车辆路径优化[J].运筹与管理,2018,27(08):20-26.
- [3] DeepSeek, DeepSeek-v4-flash, 深度求索, 2026-04-25
- [4] 张洪真,李登峰.物流配送路径选择的多目标优化模型及其字典序解法[J].物流工程与管理,2015,37(01):95-96+112.
- [5] Xiaomi, Mimo-V2.5-pro, 小米
- [6] 曹庆奎,张茜茜,任向阳.考虑动态需求的多车型应急物资配送优化研究[J].廊坊师范学院学报(自然科学版),2023,23(02):65-70+87.
- [7] 蒋映雪.突发事件下应急物流配送路径规划研究综述[J].中国物流与采购,2025,(13):35-36.DOI:10.16079/j.cnki.issn1671-6663.2025.13.009.
- [8] 张敬轩,石顺祥,李亚萍.基于蒙特卡洛概率模型的多连接体碰撞检测[J].科技创新与应用,2025,15(29):54-57.DOI:10.19981/j.CN23-1581/G3.2025.29.014.
- [9] 陈亚会,陈梦玫,迟远英.碳排放权交易的政企演化博弈与实证研究——基于技术创新与减排效果的综合视角[J/OL].北京工业大学学报(社会科学版),1-20[2026-04-26].<https://link.cnki.net/urlid/11.4558.G.20260325.1707.004>.
- [10] 谢舒婷,李金碧,邓万琼,等.基于碳减排规制的城市生鲜农产品冷链物流配送路径优化研究[J].中国市场,2026,(05):168-172.DOI:10.13939/j.cnki.zgsc.2026.05.042.

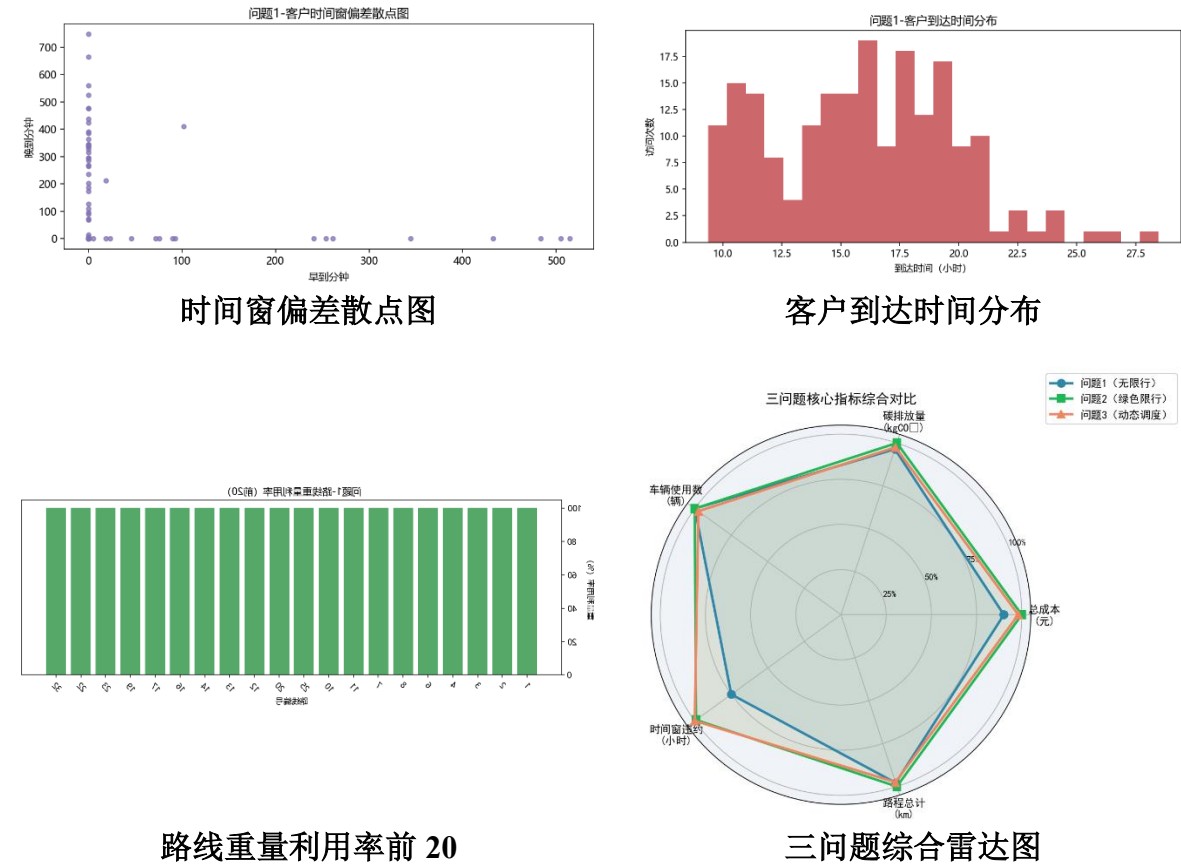
附录

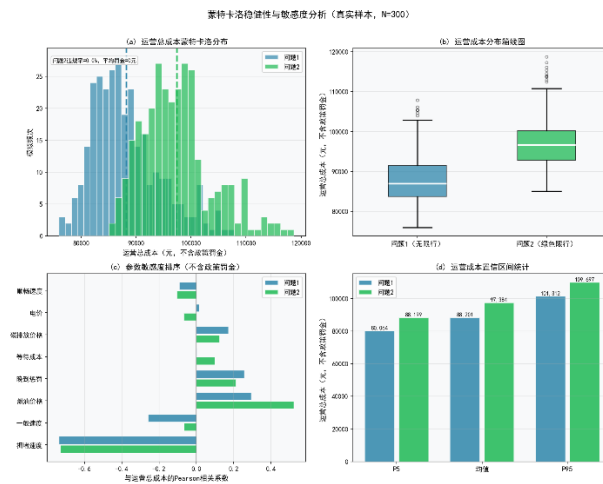
- 附录 1：支撑材料文件列表
- 附录 2：补充表格、图片
- 附录 3：AI 工具使用情况
- 附录 4：代码

附录 1： 支撑材料文件列表

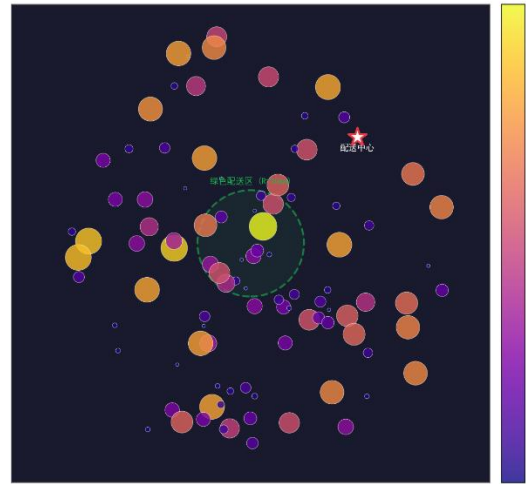
	文件名	类型	简介
问题一	data/processed	文件夹	数据预处理文件
	Script/preprocess.py	Python 代码文件	数据预处理代码
	src/pl.py	Python 代码文件	问题一代码文件
	docs/problem1_result	文件夹	问题一处理结果
问题二	src/p2.py	Python 代码文件	问题二代码文件
	docs/problem2_result	文件夹	问题二处理结果
问题三	src/p3.py	Python 代码文件	问题三代码文件
	docs/problem3_result	文件夹	问题三处理结果
灵敏度分析	src/sensitivity.py	Python 代码文件	灵敏度分析代码文件
	src/sensitivity_analysis	文件夹	灵敏度分析结果
	AI 使用情况.docx	文本文档	AI 使用说明

附录 2： 补充表格、图片

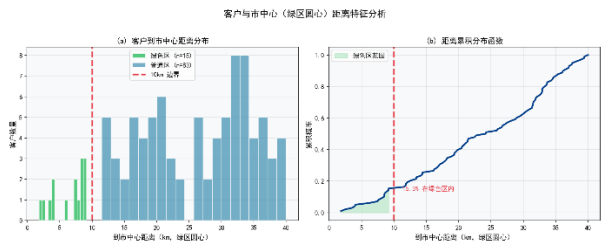




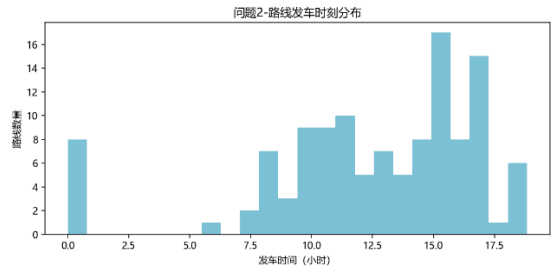
蒙特卡洛分析



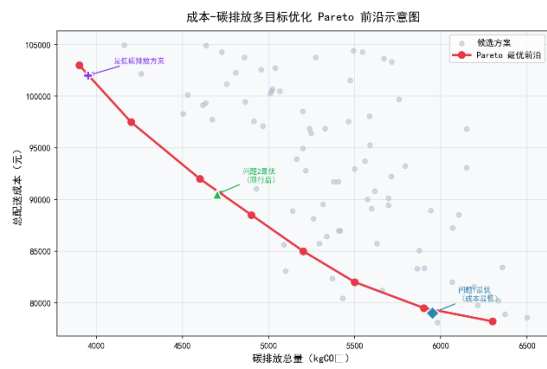
订单密度热力图



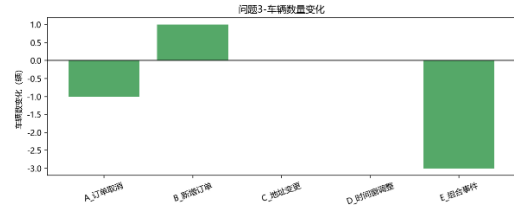
距离分布



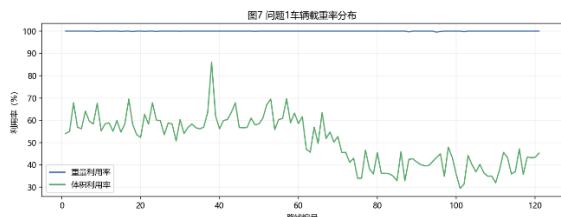
发车时刻分布



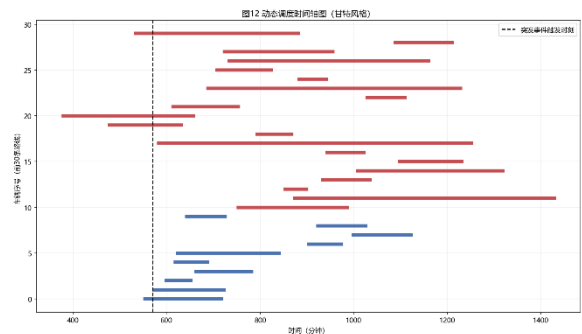
Pareto 前沿



车辆数变化



车辆载重率分布



动态调度时间轴图

AI 工具使用情况

1.问题一模型建立与求解
AI 工具版本：DeepSeek, DeepSeek-v4-flash, 深度求索, 2026-04-25
具体使用目的与环节
具体目的： 1. 验证车辆路径问题的数学模型表述是否准确 2. 优化代码绘图美观度（路径图、时间窗分布图） 3. 帮助理解速度时变模型下的行驶时间计算逻辑
具体环节： 1. 模型建立（约束条件表达） 2. 模型求解（调用求解器的参数设置） 3. 结果可视化（绘制车辆路径图）
关键交互记录（重要提示词与回复）
Query1: “我有一组客户点，每个点有软时间窗（早到等待成本 20 元/小时，晚到惩罚 50 元/小时），服务时间 20 分钟。车辆速度分时段服从正态分布。请帮我写出计算车辆从 i 到 j 行驶时间的伪代码，包含速度随机性的处理。”
Output1: 1、输入出发时间 t_{depart}，距离 $dist$ 2、判断 t_{depart} 所属时段，获取车速分布参数（均值 μ，标准差 σ） 3、随机采样车速 $v = \max(5, \text{normrnd}(\mu, \sigma))$（避免车速过低） 4、行驶时间 $\text{travel_time} = dist / v$ 5、到达时间 $t_{arrive} = t_{depart} + \text{travel_time}$ 6、若早到，增加等待成本；若晚到，增加惩罚成本。 建议多次蒙特卡洛采样取平均值以稳定结果。”
采纳和人工修改情况
采纳情况： 1. 采纳了行驶时间伪代码中的分段处理逻辑，用于编程实现。 2. 采纳了绘图代码的核心框架，包括箭头和标注。
人工修改： 1. 将速度随机采样改为取时段平均速度（因需要确定性结果），未使用蒙特卡洛平均。 2. 在绘图代码中增加了客户坐标的缩放处理，避免图形重叠。
2.问题二模型建立与求解
AI 工具版本：Xiaomi, Mimo-V2.5-pro, 小米

具体使用目的与环节
具体目的： 1. 了解动态事件下局部重调度的常用策略（如插入法、邻域搜索） 2. 辅助编写实时响应逻辑的伪代码 3. 验证突发事件对原路径可行性影响的快速判断方法 具体环节： 1. 策略设计（动态调度框架） 2. 情景模拟（订单取消/新增时的路径修复） 3. 结果展示（调整前后的路径对比图）
关键交互记录（重要提示词与回复）
Query: 配送途中突然有一个客户取消订单，该客户原本在车辆 A 的路径中。如何以最小改动重新规划剩余路径？请给出算法步骤 Output: “可采用‘删除-重插入’方法： 1、从车辆 A 的剩余任务序列中删除该客户； 2、将后续客户依次前移，重新计算到达时间及时间窗惩罚； 3、如果前移后某些客户的时间窗无法满足，则尝试将其插入到其他车辆的剩余任务中（通过贪心插入找到最小成本位置）； 4、若无可行插入，则启用备用车辆。 推荐使用局部搜索或禁忌搜索快速修复。”
采纳和人工修改情况
采纳情况： 1. 采纳了“删除-重插入”策略作为动态调度的核心方法。 2. 采纳了 Check 函数的逻辑框架，用于判断新增订单的可行性。 人工修改： 1. 在实际实现中增加了“多车协同修复”机制，不仅尝试插入原车，也尝试插入其他车辆。 2. 对 Check 函数增加了早到等待的处理（允许等待但不允许晚于时间窗上限）。

附录 4：代码

第一问部分求解代码

```
1  # -*- coding: utf-8 -*-
2  from __future__ import annotations
3
4  import argparse
5  import math
6  import random
7  from dataclasses import dataclass, field
8  from pathlib import Path
9  from typing import Dict, List, Tuple
10
11  import numpy as np
12  import pandas as pd
13  import matplotlib.pyplot as plt
14
15  from visualization_utils import safe_df_to_csv, save_fig, setup_chinese_font
16
17  WAIT_COST_PER_H = 20.0
18  LATE_COST_PER_H = 50.0
19
20  CARBON_COST_PER_KG = 0.65
21
22  FUEL_PRICE_PER_L = 7.61
23  ELECTRIC_PRICE_PER_KWH = 1.64
24
25  FUEL_CARBON_FACTOR = 2.547
26  ELECTRIC_CARBON_FACTOR = 0.501
27
28  SERVICE_MINUTES = 20.0
29  DAY_START_MIN = 8 * 60
30  DEPOT_TURNAROUND_MIN = 20.0
31
32  FLEET_OVERRUN_PENALTY = 10_000_000.0
33
34
35  @dataclass(frozen=True)
36  class VehicleType:
37      name: str
38      category: str # fuel / electric
39      cap_kg: float
40      cap_m3: float
41      count: int
42      fixed_cost: float
43      load_uplift: float # 满载修正: 燃油0.4, 新能源0.35
44
45
46  VEHICLE_TYPES: Tuple[VehicleType, ...] = (
47      VehicleType("燃油车A", "fuel", 3000.0, 13.5, 60, 400.0, 0.40),
48      VehicleType("燃油车B", "fuel", 1500.0, 10.8, 50, 400.0, 0.40),
49      VehicleType("燃油车C", "fuel", 1250.0, 6.5, 50, 400.0, 0.40),
50      VehicleType("新能源车A", "electric", 3000.0, 15.0, 10, 400.0, 0.35),
51      VehicleType("新能源车B", "electric", 1250.0, 8.5, 15, 400.0, 0.35),
52  )
53
54
55  @dataclass
56  class DemandChunk:
57      customer_id: int
58      kg: float
59      m3: float
60
61
62  @dataclass
63  class Visit:
64      customer_id: int
65      kg: float
66      m3: float
67
68
69  @dataclass
70  class Route:
```

```

71     vehicle_type: VehicleType
72     visits: List[Visit] = field(default_factory=list)
73     route_id: int = -1
74     start_minute: float = float(DAY_START_MIN)
75
76     @property
77     def total_kg(self) -> float:
78         return sum(v.kg for v in self.visits)
79
80     @property
81     def total_m3(self) -> float:
82         return sum(v.m3 for v in self.visits)
83
84     def can_hold(self, add_kg: float, add_m3: float) -> bool:
85         return (
86             self.total_kg + add_kg <= self.vehicle_type.cap_kg + 1e-9
87             and self.total_m3 + add_m3 <= self.vehicle_type.cap_m3 + 1e-9
88         )
89
90
91 @dataclass
92 class ProblemData:
93     customers: pd.DataFrame
94     dist_matrix: np.ndarray
95     tw_early: Dict[int, float]
96     tw_late: Dict[int, float]
97     demand_kg: Dict[int, float]
98     demand_m3: Dict[int, float]
99
100
101 def speed_by_time(minute_of_day: float) -> float:
102     """按分时段速度并在边界处平滑过渡 (km/h)。"""
103     t = minute_of_day % 1440.0
104     points = [
105         (0.0, 55.3), # 0:00 顺畅
106         (360.0, 35.4), # 6:00 一般
107         (480.0, 9.8), # 8:00 拥堵
108         (540.0, 55.3), # 9:00 顺畅
109         (600.0, 35.4), # 10:00 一般
110         (690.0, 9.8), # 11:30 拥堵
111         (780.0, 55.3), # 13:00 顺畅
112         (900.0, 35.4), # 15:00 一般
113         (1020.0, 9.8), # 17:00 拥堵
114         (1080.0, 35.4), # 18:00 一般
115         (1200.0, 55.3), # 20:00 顺畅
116         (1440.0, 55.3),
117     ]
118     transition = 15.0 # 边界前后各15分钟线性过渡
119     for i in range(1, len(points) - 1):
120         b, v_after = points[i]
121         v_before = points[i - 1][1]
122         if b - transition <= t < b + transition:
123             ratio = (t - (b - transition)) / (2.0 * transition)
124             return v_before + (v_after - v_before) * ratio
125     for i in range(len(points) - 1):
126         t0, v0 = points[i]
127         t1, _ = points[i + 1]
128         if t0 <= t < t1:
129             return v0
130     return points[-1][1]
131
132
133 def fuel_per_100km(v_kmh: float) -> float:
134     return 0.0025 * v_kmh * v_kmh - 0.2554 * v_kmh + 31.75
135
136
137 def electric_per_100km(v_kmh: float) -> float:
138     return 0.0014 * v_kmh * v_kmh - 0.12 * v_kmh + 36.19
139
140

```

第二问部分求解代码

```
1  # -*- coding: utf-8 -*-
2  from __future__ import annotations
3
4  import argparse
5  import math
6  import random
7  from dataclasses import dataclass, field
8  from pathlib import Path
9  from typing import Dict, List, Tuple
10
11  import numpy as np
12  import pandas as pd
13  import matplotlib.pyplot as plt
14
15  from visualization_utils import safe_df_to_csv, save_fig, setup_chinese_font
16
17
18  WAIT_COST_PER_H = 20.0
19  LATE_COST_PER_H = 50.0
20
21  CARBON_COST_PER_KG = 0.65
22
23  POLICY_VIOLATION_PENALTY = 1_000_000.0
24
25  FUEL_PRICE_PER_L = 7.61
26  ELECTRIC_PRICE_PER_KWH = 1.64
27
28  FUEL_CARBON_FACTOR = 2.547
29  ELECTRIC_CARBON_FACTOR = 0.501
30
31  SERVICE_MINUTES = 20.0
32  DAY_START_MIN = 8 * 60
33  DEPOT_TURNAROUND_MIN = 20.0
34
35  FLEET_OVERRUN_PENALTY = 10_000_000.0
36
37  GREEN_RESTRICT_START_MIN = 8 * 60
38  GREEN_RESTRICT_END_MIN = 16 * 60
39
40
41  @dataclass(frozen=True)
42  class VehicleType:
43      name: str
44      category: str # fuel / electric
45      cap_kg: float
46      cap_m3: float
47      count: int
48      fixed_cost: float
49      load_uplift: float # 满载修正: 燃油0.4, 新能源0.35
50
51
52  VEHICLE_TYPES: Tuple[VehicleType, ...] = (
53      VehicleType("燃油车A", "fuel", 3000.0, 13.5, 60, 400.0, 0.40),
54      VehicleType("燃油车B", "fuel", 1500.0, 10.8, 50, 400.0, 0.40),
55      VehicleType("燃油车C", "fuel", 1250.0, 6.5, 50, 400.0, 0.40),
56      VehicleType("新能源车A", "electric", 3000.0, 15.0, 10, 400.0, 0.35),
57      VehicleType("新能源车B", "electric", 1250.0, 8.5, 15, 400.0, 0.35),
58  )
59
60
61  @dataclass
62  class DemandChunk:
63      customer_id: int
64      kg: float
65      m3: float
66
67
68  @dataclass
69  class Visit:
70      customer_id: int
71      kg: float
72      m3: float
73
74
75  @dataclass
76  class Route:
77      vehicle_type: VehicleType
78      visits: List[Visit] = field(default_factory=list)
79      route_id: int = -1
80      start_minute: float = float(DAY_START_MIN)
81
```

```

82     @property
83     def total_kg(self) -> float:
84         return sum(v.kg for v in self.visits)
85
86     @property
87     def total_m3(self) -> float:
88         return sum(v.m3 for v in self.visits)
89
90     def can_hold(self, add_kg: float, add_m3: float) -> bool:
91         return (
92             self.total_kg + add_kg <= self.vehicle_type.cap_kg + 1e-9
93             and self.total_m3 + add_m3 <= self.vehicle_type.cap_m3 + 1e-9
94         )
95
96
97 @dataclass
98 class ProblemData:
99     customers: pd.DataFrame
100     dist_matrix: np.ndarray
101     tw_early: Dict[int, float]
102     tw_late: Dict[int, float]
103     demand_kg: Dict[int, float]
104     demand_m3: Dict[int, float]
105     green_zone_customers: set[int]
106
107
108 def speed_by_time(minute_of_day: float) -> float:
109     t = minute_of_day % 1440.0
110     points = [
111         (0.0, 55.3),
112         (360.0, 35.4),
113         (480.0, 9.8),
114         (540.0, 55.3),
115         (600.0, 35.4),
116         (690.0, 9.8),
117         (780.0, 55.3),
118         (900.0, 35.4),
119         (1020.0, 9.8),
120         (1080.0, 35.4),
121         (1200.0, 55.3),
122         (1440.0, 55.3),
123     ]
124     transition = 15.0
125     for i in range(1, len(points) - 1):
126         b, v_after = points[i]
127         v_before = points[i - 1][1]
128         if b - transition <= t < b + transition:
129             ratio = (t - (b - transition)) / (2.0 * transition)
130             return v_before + (v_after - v_before) * ratio
131     for i in range(len(points) - 1):
132         t0, v0 = points[i]
133         t1, _ = points[i + 1]
134         if t0 <= t < t1:
135             return v0
136     return points[-1][1]
137
138
139 def fuel_per_100km(v_kmh: float) -> float:
140     return 0.0025 * v_kmh * v_kmh - 0.2554 * v_kmh + 31.75
141
142
143 def electric_per_100km(v_kmh: float) -> float:
144     return 0.0014 * v_kmh * v_kmh - 0.12 * v_kmh + 36.19
145
146
147 def billable_hours(minutes: float) -> float:
148     if minutes <= 1e-9:
149         return 0.0
150     return max(1.0, minutes / 60.0)
151
152
153 def load_data(data_dir: Path) -> ProblemData:
154     customers = pd.read_csv(data_dir / "customers.csv", encoding="utf-8-sig")
155     dmat = pd.read_csv(data_dir / "distance_matrix.csv", index_col=0, encoding="utf-8-sig")
156     dist_matrix = dmat.to_numpy(dtype=float)
157
158     customers = customers.copy()
159     customers["customer_id"] = customers["customer_id"].astype(int)
160     customers["demand_kg"] = customers["demand_kg"].fillna(0.0).astype(float)
161     customers["demand_m3"] = customers["demand_m3"].fillna(0.0).astype(float)

```



```

1  def estimate_best_departure_minute(route: Route, data: ProblemData, step_min: int = 5) -> float:
2      if not route.visits:
3          return float(DAY_START_MIN)
4      first_cid = route.visits[0].customer_id
5      tw_target = data.tw_early[first_cid]
6      d = data.dist_matrix[0, first_cid]
7
8      best_t = float(DAY_START_MIN)
9      best_key = (float("inf"), float("inf"))
10     for t in range(0, 1440, max(1, step_min)):
11         v = speed_by_time(float(t))
12         arrive = float(t) + (d / max(v, 1e-6)) * 60.0
13         early = max(0.0, tw_target - arrive)
14         abs_dev = abs(arrive - tw_target)
15         key = (early, abs_dev)
16         if key < best_key:
17             best_key = key
18             best_t = float(t)
19     return best_t
20
21
22 def retime_routes(routes: List[Route], data: ProblemData) -> None:
23     for r in routes:
24         r.start_minute = estimate_best_departure_minute(r, data, step_min=5)
25
26
27 def route_policy_violations(route: Route, data: ProblemData, start_minute: float | None = None) -> int:
28     _, _, parts = evaluate_route(route, data, start_minute=start_minute)
29     return int(round(parts["policy_violations"]))
30
31
32 def repair_policy_by_retime(routes: List[Route], data: ProblemData, step_min: int = 10, max_rounds: int = 3) -> None:
33     if not routes:
34         return
35     for _ in range(max_rounds):
36         changed = False
37         for r in routes:
38             if r.vehicle_type.category != "fuel":
39                 continue
40             if not any(v.customer_id in data.green_zone_customers for v in r.visits):
41                 continue
42             cur_v = route_policy_violations(r, data)
43             if cur_v <= 0:
44                 continue
45             best_t = r.start_minute
46             best_key = (cur_v, float("inf"))
47             for t in range(0, 1440, max(1, step_min)):
48                 c, _, parts = evaluate_route(r, data, start_minute=float(t))
49                 key = (int(round(parts["policy_violations"])), c)
50                 if key < best_key:
51                     best_key = key
52                     best_t = float(t)
53                     if key[0] == 0:
54                         break
55             if abs(best_t - r.start_minute) > 1e-9:
56                 r.start_minute = best_t
57                 changed = True
58         if not changed:
59             break
60
61
62 def build_initial_solution(data: ProblemData, rng: random.Random) -> List[Route]:
63     total_kg = sum(max(0.0, v) for v in data.demand_kg.values())
64     total_m3 = sum(max(0.0, v) for v in data.demand_m3.values())
65     mix = choose_min_fleet_mix(total_kg, total_m3)
66     routes = build_routes_from_mix(mix)
67     pack_demands_into_fixed_routes(routes, data)
68
69     for r in routes:
70         rng.shuffle(r.visits)
71         r.visits = nearest_neighbor_order(r.visits, data.dist_matrix)
72     retime_routes(routes, data)
73
74     return [r for r in routes if r.visits]
75
76
77 def nearest_neighbor_order(visits: List[Visit], dist: np.ndarray) -> List[Visit]:
78     if len(visits) <= 2:
79         return visits[:]
80     remaining = visits[:]
```

```

81     ordered: List[Visit] = []
82     current = 0
83     while remaining:
84         best_idx = min(range(len(remaining)), key=lambda i: dist[current, remaining[i].customer_id])
85         nxt = remaining.pop(best_idx)
86         ordered.append(nxt)
87         current = nxt.customer_id
88     return ordered
89
90
91 def best_insertion_position(route: Route, visit: Visit, dist: np.ndarray) -> Tuple[int, float]:
92     n = len(route.visits)
93     if n == 0:
94         inc = dist[0, visit.customer_id] + dist[visit.customer_id, 0]
95         return 0, float(inc)
96     best_pos = 0
97     best_inc = float("inf")
98     for pos in range(n + 1):
99         if pos == 0:
100             prev_node = 0
101             next_node = route.visits[0].customer_id
102         elif pos == n:
103             prev_node = route.visits[-1].customer_id
104             next_node = 0
105         else:
106             prev_node = route.visits[pos - 1].customer_id
107             next_node = route.visits[pos].customer_id
108         inc = dist[prev_node, visit.customer_id] + dist[visit.customer_id, next_node] - dist[prev_node, next_node]
109         if inc < best_inc:
110             best_inc = float(inc)
111             best_pos = pos
112     return best_pos, best_inc
113
114
115 def compress_routes(routes: List[Route], data: ProblemData, max_passes: int = 4) -> List[Route]:
116     if not routes:
117         return routes
118     work = copy_routes(routes)
119     for _ in range(max_passes):
120         changed = False
121         # 优先处理“访问少且载重轻”的路线
122         order = sorted(
123             range(len(work)),
124             key=lambda i: (len(work[i].visits), work[i].total_kg + 0.3 * work[i].total_m3 * 1000.0),
125         )
126         removed_indices = set()
127         for src_idx in order:
128             if src_idx in removed_indices or src_idx >= len(work):
129                 continue
130             src = work[src_idx]
131             if not src.visits:
132                 removed_indices.add(src_idx)
133                 changed = True
134                 continue
135
136             temp_targets = copy_routes(work)
137             feasible_all = True
138             for v in list(src.visits):
139                 best_dst = -1
140                 best_pos = 0
141                 best_score = float("inf")
142                 for dst_idx, dst in enumerate(temp_targets):
143                     if dst_idx == src_idx:
144                         continue
145                     if not dst.can_hold(v.kg, v.m3):
146                         continue
147                     pos, inc = best_insertion_position(dst, v, data.dist_matrix)
148                     if inc < best_score:
149                         best_score = inc
150                         best_dst = dst_idx
151                         best_pos = pos
152                 if best_dst < 0:
153                     feasible_all = False
154                     break

```

第三问部分求解代码

```
1  # -*- coding: utf-8 -*-
2  from __future__ import annotations
3
4  import argparse
5  import copy as copy_module
6  import sys
7  import time
8  from dataclasses import dataclass, field
9  from pathlib import Path
10 from typing import Dict, List, Optional, Tuple
11
12 import numpy as np
13 import pandas as pd
14 import matplotlib.pyplot as plt
15 from visualization_utils import safe_df_to_csv, save_fig, setup_chinese_font
16
17 sys.path.insert(0, str(Path(__file__).resolve().parent))
18 from problem2_solver import ( # noqa: E402
19     assign_route_trips_to_physical_slots,
20     DAY_START_MIN,
21     ELECTRIC_CARBON_FACTOR,
22     ELECTRIC_PRICE_PER_KWH,
23     FUEL_CARBON_FACTOR,
24     FUEL_PRICE_PER_L,
25     GREEN_RESTRICT_END_MIN,
26     GREEN_RESTRICT_START_MIN,
27     LATE_COST_PER_H,
28     POLICY_VIOLATION_PENALTY,
29     SERVICE_MINUTES,
30     VEHICLE_TYPES,
31     WAIT_COST_PER_H,
32     ProblemData,
33     Route,
34     Visit,
35     VehicleType,
36     best_insertion_position,
37     billable_hours,
38     compress_routes,
39     copy_routes,
40     electric_per_100km,
41     estimate_best_departure_minute,
42     evaluate_route,
43     evaluate_solution,
44     fuel_per_100km,
45     is_in_restricted_window,
46     load_data,
47     nearest_neighbor_order,
48     remove_empty_routes,
49     repair_policy_by_retime,
50     retime_routes,
51     speed_by_time,
52 )
53
54 @dataclass
55 class RouteState:
56     """事件触发时刻 t_event 下，单条路线的快照状态。"""
57
58     route: Route
59     completed_visits: List[Visit] # 已完成（冻结），不可修改
60     completed_arrivals: List[float] # 各已完成访问的到达时刻
61     remaining_visits: List[Visit] # 未完成（激活），可重规划
62     current_node: int # 上一个已完成服务的节点（0=仓库）
63     current_time: float # 在 current_node 完成服务后的时刻
64     in_transit_to: int # >0=正在前往/服务中；-1=空闲/全部完成
65
66     @property
67     def has_remaining(self) -> bool:
68         return bool(self.remaining_visits) or self.in_transit_to > 0
69
70
71 @dataclass
72 class DynamicEvent:
73     event_type: str # "cancel" | "new_order" | "address_change" | "tw_adjust"
74     trigger_time: float # 事件触发时刻（当日分钟数）
75     customer_id: int # 主要受影响的客户 ID
```

```

76     description: str = ""
77     new_demand_kg: float = 0.0
78     new_demand_m3: float = 0.0
79     adjusted_tw_early: float = -1.0
80     adjusted_tw_late: float = -1.0
81
82
83 @dataclass
84 class ScenarioResult:
85     name: str
86     event: DynamicEvent
87     before_cost: float
88     after_cost: float
89     cost_delta: float
90     before_detail: Dict
91     after_detail: Dict
92     before_n_routes: int
93     after_n_routes: int
94     response_ms: float
95     event_log: List[str]
96
97
98 def load_p2_routes(p2_result_dir: Path, data: ProblemData) -> List[Route]:
99     routes_df = pd.read_csv(p2_result_dir / "routes.csv", encoding="utf-8-sig")
100     arrivals_df = pd.read_csv(p2_result_dir / "arrivals_raw.csv", encoding="utf-8-sig")
101     by_name = {v.name: v for v in VEHICLE_TYPES}
102
103     routes: List[Route] = []
104     for _, row in routes_df.iterrows():
105         rid = int(row["route_id"])
106         start_min = float(row["start_minute"])
107         vtype = by_name[str(row["vehicle_type"])]
108
109         arr_sub = arrivals_df[arrivals_df["route_id"] == rid].sort_values("seq")
110         visits = [
111             Visit(
112                 customer_id=int(row["customer_id"]),
113                 kg=float(row["deliver_kg"]),
114                 m3=float(row["deliver_m3"]),
115             )
116             for _, row in arr_sub.iterrows()
117         ]
118         routes.append(
119             Route(vehicle_type=vtype, visits=visits, route_id=rid, start_minute=start_min)
120         )
121
122     return routes
123
124
125 def split_route_at_time(
126     route: Route, t_event: float, data: ProblemData
127 ) -> RouteState:
128     completed: List[Visit] = []
129     completed_arrivals: List[float] = []
130     current_node = 0
131     current_time = route.start_minute
132
133     for i, visit in enumerate(route.visits):
134         d = data.dist_matrix[current_node, visit.customer_id]
135         speed = speed_by_time(current_time)
136         travel_min = d / max(speed, 1e-6) * 60.0
137         arrive_time = current_time + travel_min
138
139         if arrive_time >= t_event:
140             # 正在前往此节点
141             return RouteState(
142                 route=route,
143                 completed_visits=completed,
144                 completed_arrivals=completed_arrivals,
145                 remaining_visits=list(route.visits[i:]),
146                 current_node=current_node,
147                 current_time=current_time,
148                 in_transit_to=visit.customer_id,
149             )

```